

ModNet: A Self-Growing Layer-Free Recurrent Network for Universal Computation

An evolutionary framework that discovers structure from first principles

Masoud Azizi¹

Department of Computer Science
Faculty of Mathematical Sciences
University of Tabriz
Tabriz, Iran

September 17, 2026

¹Corresponding author. Email: mablue92@gmail.com

Abstract

We present MODNET, a self-growing layer-free recurrent network whose topology is discovered through evolution rather than imposed by a designer. The architecture has no predefined layers and no modular grouping: every node is a general-purpose threshold unit that may receive input from any other node, including itself, and the first n_{out} nodes of the network are designated as outputs. Starting from a minimal random topology, the network grows, prunes, and reorganizes itself through a compact evolutionary loop.

We evaluate MODNET on a suite of sixteen benchmark problems spanning Boolean logic, digital arithmetic, sorting networks, and continuous regression. The network solves fourteen of the sixteen tasks, including full adder, 2-bit adder, comparator, 4-to-1 multiplexer, and 4-bit sorting network, all with exact 100% accuracy using no more than 27 nodes. On continuous targets, MODNET achieves root-mean-square errors as low as 0.0143 on the $x^2 + y^2$ surface and 0.0254 on the multiplication task with only 13 nodes. We observe that recurrence emerges spontaneously and scales with task complexity: XOR-2 requires one self-loop, while Gaussian and multiplication require 10 to 14 self-loops, indicating that temporal memory is a structural necessity, not an imposed design.

Our results suggest that a fully layer-free architecture driven by a minimal evolutionary process can rediscover structural principles — non-linearity, recurrence, sparsity — that are typically imposed by human designers. We provide a complete open-source implementation and a reproducible benchmark suite.

Keywords: neuroevolution, layer-free architectures, self-growing networks, recurrent networks, Boolean function learning, digital circuits, sparse connectivity.

Contents

1	Introduction	8
1.1	Motivation	8
1.2	Background	8
1.2.1	The XOR Problem and the Limits of Linear Models	8
1.2.2	Evolutionary Approaches to Network Design	9
1.2.3	Modularity in Neural Networks	9
1.2.4	Sparse Connectivity	9
1.2.5	Universal Computation	10
1.3	The Present Work	10
1.4	Key Findings	11
1.4.1	Recurrence Emerges, and Scales with Complexity	11
1.4.2	Sparsity Is the Norm	11
1.4.3	Growth Adapts to Task Complexity	12
1.4.4	Layer-Free and Modular Architectures Are Not Equivalent	12
1.5	Contributions	12
1.6	Organization	13
2	Related Work	14
2.1	Neuroevolution: Evolving Network Topologies	14
2.2	Growing and Self-Organizing Networks	15
2.2.1	Constructive Growth	15
2.2.2	Developmental Systems	15
2.2.3	Neural Cellular Automata	16
2.3	Modular Neural Networks	16
2.4	Sparse Connectivity	17
2.5	Neural Networks for Digital Circuits	17
2.6	Universal Computation	18
2.7	Biological Inspiration: Criticality and Small-World Topology	18
2.8	Neural Architecture Search	18
2.9	Summary and Positioning	19
3	Method	20
3.1	Design Principles	20
3.2	Node Model	21
3.3	Layer-Free Connectivity	21
3.3.1	Output Selection	21
3.4	Recurrence	22
3.5	Fitness Evaluation	22
3.5.1	Boolean tasks	22

3.5.2	Regression tasks	23
3.6	Evolutionary Algorithm	23
3.6.1	Mutation	23
3.6.2	Crossover	24
3.6.3	Birth	24
3.6.4	Pruning	25
3.6.5	Selection	25
3.7	Complexity Analysis	25
3.8	Summary of Hyperparameters	26
3.9	Reproducibility	26
4	Experimental Setup	27
4.1	Benchmark Problems	27
4.1.1	Boolean Function Learning	27
4.1.2	Digital Circuits	28
4.1.3	Continuous Regression	28
4.2	Data Encoding	29
4.2.1	Boolean Inputs	29
4.2.2	Continuous Inputs	29
4.2.3	Outputs	29
4.3	Evaluation Procedure	29
4.4	Hyperparameter Settings	30
4.5	Network Visualization	31
4.6	Reproducibility	32
4.7	Computational Cost	32
4.8	Summary of Reproducibility Artifacts	33
5	Results	34
5.1	Overview	34
5.2	Boolean Function Learning	35
5.2.1	XOR-2	35
5.2.2	XOR-3	35
5.2.3	Parity-4	35
5.2.4	Parity-6	36
5.2.5	Majority-3 and Majority-5	36
5.3	Digital Circuits and Sorting	37
5.3.1	Full Adder	37
5.3.2	2-bit Adder	37
5.3.3	Comparator	37
5.3.4	MUX 4-to-1	37
5.3.5	Sort-4	37
5.4	Continuous Regression	39
5.4.1	$\sin(2\pi x)$	39
5.4.2	$\sin(4\pi x)$	39
5.4.3	$x^2 + y^2$	39
5.4.4	Gaussian	39
5.4.5	Multiplication	40
5.5	Emergent Structural Properties	43
5.5.1	Recurrence Scales with Complexity	43

5.5.2	Sparsity Is the Norm	43
5.5.3	Growth Adapts to Task Complexity	44
5.6	Failure Analysis	44
5.6.1	Parity-6	44
5.6.2	$\sin(4\pi x)$	45
5.7	Summary of Key Findings	46
6	Discussion	47
6.1	Interpretation of the Main Findings	47
6.1.1	Layer-Free Computation Is Feasible for Digital Logic	47
6.1.2	Recurrence Emerges and Scales with Task Complexity	47
6.1.3	Layer-Free and Modular Architectures Are Not Equivalent	48
6.1.4	Sparsity Is the Norm	49
6.1.5	Growth Adapts to Task Complexity	49
6.2	Relationship to Related Work	49
6.2.1	Comparison with NEAT and HyperNEAT	49
6.2.2	Comparison with Self-Organizing Systems	50
6.2.3	Comparison with Differentiable Sorting	50
6.3	Limitations	51
6.3.1	Limited Scale	51
6.3.2	Limited Task Diversity	51
6.3.3	Two Failure Cases	51
6.3.4	No Theoretical Guarantees	51
6.3.5	Single Seed	52
6.4	Threats to Validity	52
6.4.1	Internal Validity	52
6.4.2	External Validity	52
6.4.3	Construct Validity	52
6.5	Hypotheses for the Parity-6 Failure	52
6.5.1	Hypothesis 1: Insufficient Time Steps	53
6.5.2	Hypothesis 2: Local Optimum with Under-Parameterization	53
6.5.3	Hypothesis 3: Insufficient Population Diversity	53
6.6	Future Work	53
6.6.1	Scaling to Larger Networks	53
6.6.2	Multi-Seed Analysis	53
6.6.3	Extending to Structured Inputs	54
6.6.4	Combining Evolution with Gradient Descent	54
6.6.5	Theoretical Analysis	54
6.7	Broader Implications	55
6.8	Summary	55
7	Conclusion	57
7.1	Summary of the Work	57
7.2	What We Learned	58
7.2.1	Structure Is Discoverable	58
7.2.2	Recurrence Is a Structural Necessity, Not a Design Choice	58
7.2.3	Sparsity Is Preferred	59
7.2.4	Modularity Is Not Necessary	59
7.2.5	Digital Circuits Are Within Reach	59

7.3	Limitations	59
7.3.1	Scale	59
7.3.2	Two Failure Cases	59
7.3.3	No Theoretical Guarantees	60
7.3.4	Single Seed	60
7.3.5	Limited Task Diversity	60
7.4	Future Work	60
7.4.1	Scaling to Larger Networks	60
7.4.2	Hybrid Learning	60
7.4.3	Extending to Structured Inputs	60
7.4.4	Theoretical Analysis	61
7.5	Broader Implications	61
7.6	Final Remarks	61
A	Hyperparameter Summary	63
B	Network Snapshots	64
C	Algorithm Details	65

List of Figures

3.1	Node model. Each node computes a weighted sum of its sources plus a bias, then applies a Heaviside step function. The self-loop indicates that the node may receive input from itself, which introduces recurrence.	21
3.2	Pruning removes nodes whose activity is either below α_{lo} or above α_{hi} across all training samples (dashed gray nodes). The remaining nodes and connections are re-indexed to form a compact network.	25
4.1	Spring-layout visualization of the XOR-2 network. Green squares are input ports, blue circles are hidden nodes, red circles are output nodes. The single self-loop is shown as a red arc. The network has 8 nodes and 17 active connections.	31
5.1	The final network evolved for XOR-2, shown in spring (force-directed) layout. Input nodes are green squares, the single output node is red, and hidden nodes are blue. Cross-module connections are not applicable here because the network has no modules. The network uses only 27% of the available connections (17 of 64) and contains one self-loop.	35
5.2	Evolution of the XOR-3 network across three snapshots. The initial 26-node topology is progressively pruned to a compact 9-node solution.	36
5.3	The final network evolved for Parity-4. Recurrence is present in 5 of the 10 nodes.	36
5.4	The best network evolved for Parity-6. It reached 95.3% accuracy but did not reach the exact-solution threshold. The network has 8 nodes and 5 recurrent self-loops.	37
5.5	The final network evolved for the full adder. Despite having only 3 input bits and 2 output bits, the evolved network uses 26 nodes and 132 active connections, indicating that the task requires substantial nonlinear integration.	38
5.6	The final network evolved for the 4-bit sorting network. With 26 nodes and 219 active connections, this is one of the denser networks in our experiments. Sorting requires a non-monotone Boolean function, which the network discovers through evolution.	39
5.7	The final network evolved for $\sin(2\pi x)$. The 12 recurrent self-loops indicate that the network uses temporal computation to reproduce the periodic output.	40
5.8	The final network evolved for $x^2 + y^2$. Despite having the largest input dimension (8 bits, encoding two continuous variables), this task was solved in fewer generations than $\sin(4\pi x)$	41
5.9	The final network evolved for the Gaussian task. The 14 recurrent self-loops are the highest in the entire suite.	41
5.10	The final network evolved for $x_1 \cdot x_2$. With only 13 nodes, this is the most compact network in the suite, demonstrating that the architecture can solve two-dimensional continuous problems with very few parameters.	42

- 5.11 Learning curve for Parity-6. The fitness improves from -0.50 to -0.0469 over 1500 generations but does not reach zero. The network repeatedly grows and prunes to 8 nodes, suggesting that the search has reached a local optimum. . . . 45
- 5.12 Learning curve for $\sin(4\pi x)$. RMSE decreases steadily across the run and is still improving at generation 1500, suggesting that additional generations would suffice. 46

List of Tables

1.1	Headline results of MODNET on the sixteen benchmark problems. Boolean tasks are measured in exact accuracy; continuous tasks in root-mean-square error (RMSE). Full details are reported in Chapter 5.	11
2.1	Positioning of MODNET relative to related architectures. “Layer-free” means no predefined layers or grid substrate. “No modules” means no explicit modular grouping. “Grown” means both addition and pruning of nodes. “Gradient-free” means no gradient information at any stage.	19
3.1	Hyperparameters of MODNET.	26
4.1	Summary of the sixteen benchmark problems.	28
4.2	Hyperparameters of MODNET used in all experiments.	30
4.3	Task-specific hyperparameters. T is the number of time steps per evaluation, n_0 is the initial number of nodes, $G_{\max}$ is the maximum number of generations. . . .	30
4.4	Wall-clock time per task, sorted by ascending time.	32
4.5	Reproducibility artifacts per task.	33
5.1	Summary of experimental results on the sixteen benchmark problems. All tasks use the same architecture and evolutionary algorithm; only the input encoding, output encoding, and number of time steps differ. N : nodes in the final network. R : recurrent self-loops. E : active connections. Numbers in bold indicate that the task met its success threshold.	34
5.2	Number of recurrent self-loops per task. The count is sorted in ascending order. Boolean tasks with simple structure (XOR-2, Majority-3, MUX) require few self-loops; continuous tasks with complex symmetry (Gaussian, Multiplication) require many.	43
5.3	Density of evolved networks. Density = E/N^2	44
6.1	Recurrence count scales with task complexity. XOR-2, a purely combinatorial problem, requires a single self-loop. Gaussian and multiplication, both continuous, require ten to fourteen.	48
6.2	Comparison of layer-free and modular variants. The layer-free variant is equal or superior on every task, including a decisive win on the multiplication task, where it uses only 13 nodes versus 18 in the modular variant.	48
7.1	Headline results of MODNET on the sixteen benchmark problems. Boolean tasks are measured in exact accuracy; continuous tasks in root-mean-square error (RMSE).	58
A.1	Complete list of hyperparameters for MODNET.	63

Chapter 1

Introduction

1.1 Motivation

The design of artificial neural networks has historically been guided by human intuition. From the perceptron model of Rosenblatt [52] to modern deep architectures [30], researchers have manually specified the number of layers, the connectivity patterns, the activation functions, and the learning rules. This top-down approach has produced remarkable results, but it has also imposed structural constraints that may not be optimal for every task.

A central example of such a constraint is the layered architecture itself. Since the backpropagation algorithm was popularized by Rumelhart et al. [53], nearly all successful neural networks have been organized into discrete layers. This design is computationally convenient—it allows efficient matrix multiplication and gradient computation—but it also imposes a rigid separation between “input,” “hidden,” and “output” units. In a biological brain, by contrast, there are no layers. Neurons form a dense, recurrent graph in which any neuron may connect to any other, and structure emerges through developmental and evolutionary processes rather than explicit design.

This discrepancy raises a fundamental question:

Can a neural network discover its own structure from first principles, without a designer specifying layers, connectivity, or even the number of nodes?

The present work answers this question in the affirmative, through a minimal architecture we call MODNET.

1.2 Background

1.2.1 The XOR Problem and the Limits of Linear Models

The limitations of fixed, single-layer architectures were formalized by Minsky and Papert [34], who proved that a single-layer perceptron cannot compute the XOR function, nor the parity function for any number of inputs. This result, often referred to as the “XOR problem,” is not merely a technical curiosity. It reflects a deeper mathematical fact: linear threshold units can only separate linearly separable classes, and parity is not linearly separable for any $n \geq 2$.

The XOR problem had a profound impact on the field. It contributed to the first “AI winter” and demonstrated that architectural choices—in particular, the presence or absence of hidden layers—are not merely implementation details but determine the computational power of the model. It was not until the rediscovery of backpropagation by Rumelhart et al. [53] that

multi-layer networks became trainable, and the XOR problem was finally solved by adding a single hidden layer.

1.2.2 Evolutionary Approaches to Network Design

An alternative to gradient-based training is to evolve the network topology itself. The NeuroEvolution of Augmenting Topologies (NEAT) algorithm, introduced by Stanley and Miikkulainen [62], was a landmark contribution in this direction. NEAT starts with a minimal network and incrementally adds nodes and connections, using a principled crossover mechanism and speciation to protect structural innovation. It demonstrated that evolving topology alongside weights can significantly accelerate learning on reinforcement learning benchmarks.

NEAT was later extended by HyperNEAT [63], which uses an indirect encoding called a Compositional Pattern-Producing Network (CPPN) to generate connectivity patterns with geometric regularity. HyperNEAT enabled the evolution of much larger networks than direct encoding, but it still relies on a predefined substrate geometry, typically a grid of neurons.

A parallel line of work explored growing neural networks with incremental learning. The Cascade-Correlation architecture of Fahlman and Lebiere [13] begins with a minimal network and adds hidden units one by one. The Growing Neural Gas algorithm of Fritzke [14] uses competitive Hebbian learning to build topological representations incrementally. Both approaches demonstrate the power of growth, but neither fully addresses the problem of recurrence, nor the possibility of a fully layer-free topology.

1.2.3 Modularity in Neural Networks

Modularity—the decomposition of a system into cohesive, loosely coupled components—is a well-established principle in software engineering [41]. Alho et al. [1] conducted a systematic review of 86 studies on modular neural networks and found that nearly two-thirds report improvements in task accuracy compared to monolithic solutions.

In the context of neuroevolution, Khare et al. [26] proposed a co-evolutionary model for automatic problem decomposition, in which a population of modules is evolved alongside a population of complete systems that combine modules. They demonstrated that if a particular task decomposition is beneficial, it can be evolved through this mechanism. However, their model requires the designer to specify the number of modules and the interface between them.

Landassuri-Moreno [28] investigated the emergence of modularity in evolved neural networks and found that pure-modular architectures emerge at low connectivity values. These results suggest that modularity is not merely a design choice but can be an emergent property of evolutionary search.

In the present work, we deliberately *abandon* explicit modular structure. Our architecture has no modules at all. This is a stronger demonstration of emergence: the properties we observe—recurrence, sparsity, cross-scale integration—must arise entirely from the evolutionary process, without any modular scaffold to support them.

1.2.4 Sparse Connectivity

Biological neural networks are highly sparse: the probability that any two neurons are connected is on the order of 10^{-6} [30]. This sparsity is not accidental; it is a consequence of metabolic constraints and is believed to be essential for efficient information processing [8].

In artificial neural networks, sparsity has been studied primarily as a compression technique. The Sparse Evolutionary Training (SET) method of Mocanu et al. [35] evolves an initial sparse topology into a scale-free topology during training, allowing the training of networks with over

one million neurons on commodity hardware. However, these methods typically begin with a predefined network architecture and prune or sparsify it. They do not address the question of how sparsity and connectivity structure emerge from first principles.

1.2.5 Universal Computation

The theoretical foundations of neural computation were established by Siegelmann and Sontag [57], who proved that recurrent neural networks with rational weights are Turing complete. More recently, Pérez et al. [42] showed that transformers and Neural GPUs are also Turing complete, without requiring external memory. These results establish that neural networks can, in principle, simulate any computable function, but they do not address the question of how such networks can be discovered or evolved.

1.3 The Present Work

In this paper, we present MODNET, a self-growing layer-free recurrent network that addresses the limitations of fixed architectures by discovering its own structure through evolution. The architecture has four defining characteristics:

1. **Layer-free.** There are no predefined layers, and no modular grouping. Every node is a general-purpose threshold unit that may connect to any other node, including itself.
2. **Outputs as ordinary nodes.** The first n_{out} nodes of the network are designated as outputs; there is no separate output layer.
3. **Grown and pruned.** The network starts from a small random topology and grows through addition of nodes and pruned through removal of inactive or unreachable ones. Growth is bidirectional.
4. **Gradient-free.** No gradient information is used at any stage. All learning is performed by a compact evolutionary algorithm operating on the topology and weights simultaneously.

We evaluate MODNET on a suite of **sixteen** benchmark problems spanning four broad classes: Boolean logic, digital arithmetic, sorting networks, and continuous regression. Table 1.1 summarizes the headline results.

Three observations follow immediately from Table 1.1. First, MODNET solves **fourteen of sixteen** tasks to their success threshold, including all of the digital arithmetic and sorting tasks. Second, the evolved networks are small: no task required more than 27 nodes, and the multiplication task required only 13. Third, the same architecture, without any task-specific modification, handles Boolean logic, digital arithmetic, sorting, and continuous regression.

Table 1.1: Headline results of MODNET on the sixteen benchmark problems. Boolean tasks are measured in exact accuracy; continuous tasks in root-mean-square error (RMSE). Full details are reported in Chapter 5.

Task	Type	Result	Nodes	Time (s)
XOR-2	Boolean	100.0%	8	0.8
XOR-3	Boolean	100.0%	9	2.0
Parity-4	Boolean	100.0%	10	1.8
Parity-6	Boolean	95.3%	8	40.6
Majority-3	Boolean	100.0%	26	0.9
Majority-5	Boolean	100.0%	26	10.2
Full Adder	Boolean	100.0%	26	3.0
2-bit Adder	Boolean	100.0%	26	7.3
Comparator	Boolean	100.0%	27	3.8
MUX 4-to-1	Boolean	100.0%	26	4.8
Sort-4	Boolean	100.0%	26	8.7
$\sin(2\pi x)$	Regression	RMSE = 0.0465	18	30.0
$\sin(4\pi x)$	Regression	RMSE = 0.0789	13	25.1
$x^2 + y^2$	Regression	RMSE = 0.0143	26	21.1
Gaussian	Regression	RMSE = 0.0275	27	41.6
Multiplication	Regression	RMSE = 0.0254	13	28.5

1.4 Key Findings

Beyond the raw numbers, we observe several structural properties that emerge spontaneously from the evolutionary process.

1.4.1 Recurrence Emerges, and Scales with Complexity

Every successfully evolved network contains recurrent self-loops, ranging from one self-loop (XOR-2) to fourteen (Gaussian). No mechanism in our algorithm explicitly favors recurrence. Self-loops emerge because the unrestricted topology allows them, and evolutionary pressure discovers them to be useful.

Crucially, the number of self-loops scales with the temporal complexity of the task. XOR-2, a purely combinatorial problem, requires only one self-loop. Gaussian and multiplication, both continuous, require ten to fourteen. This suggests that recurrence is a *structural necessity* for tasks that require temporal integration, and that MODNET discovers this necessity without being told.

1.4.2 Sparsity Is the Norm

Evolved networks use a small fraction of the available connections. Density, defined as the ratio of active connections to total possible connections (E/N^2), ranges from 27% to 67% depending on the task. This sparsity was not imposed by any regularizer; it emerged from the pruning mechanism, which removes nodes whose activity is either too low or too high.

The observed sparsity is consistent with that of biological neural networks [8], where the vast majority of potential synapses are absent.

1.4.3 Growth Adapts to Task Complexity

The initial network has 26 nodes. Over evolution, networks either shrank, grew, or stayed approximately the same:

- **Shrinking:** XOR-2 ($26 \rightarrow 8$), XOR-3 ($26 \rightarrow 9$), Parity-4 ($26 \rightarrow 10$), Multiplication ($26 \rightarrow 13$).
- **Growing:** $\sin(2\pi x)$ ($26 \rightarrow 18$), Gaussian ($26 \rightarrow 27$).
- **Nearly stable:** Parity-6, Majority-5, Full Adder.

Boolean tasks tend to converge to small networks, while continuous tasks require larger networks. The pattern is consistent with the intuition that Boolean functions have low algorithmic complexity, while continuous functions require more expressive approximations.

1.4.4 Layer-Free and Modular Architectures Are Not Equivalent

In earlier work we studied a modular variant of MODNET, where nodes were grouped into a fixed number of modules. The present work is fully layer-free: there are no modules. A direct comparison reveals that the layer-free variant is *equal or superior* on every benchmark, including a decisive win on the multiplication task, where it uses only 13 nodes versus 18 in the modular variant.

This suggests that modular scaffolds, far from helping, can *restrict* the search when the task does not require modular structure.

1.5 Contributions

The main contributions of this work are:

1. **Architecture.** We introduce a fully layer-free, recurrent network architecture with no modular grouping and no separate output layer. The entire structure, including the number of nodes, is discovered through evolution.
2. **Comprehensive benchmark suite.** We evaluate on **sixteen** tasks spanning Boolean logic, digital arithmetic, sorting networks, and continuous regression. To the best of our knowledge, this is the first time a single layer-free evolved architecture has been demonstrated on such a range of tasks.
3. **Digital circuit results.** We show that MODNET solves full adder, 2-bit adder, comparator, 4-to-1 multiplexer, and 4-bit sorting network, all with exact 100% accuracy. This establishes that evolved layer-free networks can implement digital logic of nontrivial complexity.
4. **Emergent recurrence.** We observe that recurrence emerges spontaneously and scales with the temporal complexity of the task. XOR-2 requires one self-loop; Gaussian requires fourteen.
5. **Comparison with modular variant.** We compare against a modular variant of the same algorithm, and find the layer-free variant equal or superior on every benchmark.
6. **Reproducibility.** We provide a complete, open-source implementation and a reproducible benchmark suite, including per-task graphs in force-directed (spring) layout, event logs of every birth and prune operation, and full training curves.

1.6 Organization

The remainder of this thesis is organized as follows.

Chapter 2 reviews related work in neuroevolution, growing networks, modular neural networks, sparse training, and universal computation.

Chapter 3 describes the MODNET architecture in detail, including the node model, the layer-free connectivity, the recurrence mechanism, the fitness function, and the evolutionary algorithm.

Chapter 4 presents the experimental setup: the sixteen benchmark problems, the data encoding, the evaluation procedure, and the hyperparameters.

Chapter 5 reports the results, including the summary table, per-task network graphs in spring layout, learning curves, and a detailed analysis of the two failure cases (Parity-6 and $\sin(4\pi x)$).

Chapter 6 interprets the findings, discusses limitations and threats to validity, and proposes directions for future work.

Chapter 7 concludes the thesis.

Appendix A lists all hyperparameters. **Appendix B** presents network snapshots at selected generations. **Appendix C** provides the complete algorithm in pseudocode.

Chapter 2

Related Work

Our work sits at the intersection of six research traditions: neuroevolution, growing and self-organizing networks, modular neural networks, sparse connectivity, digital circuit synthesis with neural networks, and universal computation. This chapter reviews each in turn and identifies the gap that MODNET addresses.

2.1 Neuroevolution: Evolving Network Topologies

The idea that a neural network’s topology can be evolved rather than designed dates back to the early 1990s. Gruau [18] introduced *Cellular Encoding*, in which a genetic program simultaneously evolves the architecture, weights, thresholds, and biases of a neural network. The genotype is a tree of graph-rewriting rules inspired by L-systems; through development, this tree unfolds into a network of arbitrary size and shape. Gruau demonstrated that cellular encoding could generate modular neural networks and used it to evolve a controller for a hexapod robot. This early work established the principle that complex structure can emerge from a compact generative description.

The NeuroEvolution of Augmenting Topologies (NEAT) algorithm of Stanley and Miikkulainen [62] became the canonical framework for topology-evolving neuroevolution. NEAT starts from a minimal network and incrementally adds nodes and connections through structural mutation. It introduces three key ideas:

1. **Historical markings** to allow meaningful crossover between networks of different topologies.
2. **Speciation** to protect structural innovation from premature competition.
3. **Complexification** from small initial topologies.

NEAT was shown to outperform fixed-topology methods on reinforcement learning benchmarks [61] and has since been applied to domains ranging from autonomous rover navigation [56] to scheduling problems [54].

HyperNEAT [63] extended NEAT with an indirect encoding based on Compositional Pattern-Producing Networks (CPPNs). Instead of directly encoding each connection, HyperNEAT evolves a CPPN whose output at any pair of coordinates specifies the weight of the corresponding connection. This geometric encoding exploits regularity in the substrate and scales to networks with millions of connections [9]. The Enhanced Hypercube-based Encoding (ES-HyperNEAT) of Risi and Stanley [49] further allowed the placement, density, and connectivity of neurons to be evolved rather than fixed.

Recent work has focused on accelerating NEAT and extending its reach. Wang et al. [69] introduced a tensorization method that transforms the diverse topologies of NEAT into uniformly shaped tensors, enabling GPU acceleration and reducing training time by an order of magnitude. Lyubchev and Marchev [33] revisited the original NEAT formalism and proposed improvements to the mutation and selection operators.

Despite their success, these methods share a common assumption: the network is organized into layers or a predefined substrate geometry. HyperNEAT’s CPPN operates on a grid; NEAT’s complexification adds nodes to hidden layers; tensorized NEAT pads topologies into fixed-shape tensors. None of these approaches explores fully layer-free computation in which *any node may connect to any other node*, including itself, and in which there is no modular scaffold.

2.2 Growing and Self-Organizing Networks

An alternative to evolving a fixed topology is to grow the network through local rules, either developmental or activity-dependent.

2.2.1 Constructive Growth

The Cascade-Correlation architecture of Fahlman and Lebiere [13] begins with a minimal network and adds hidden units one by one, training each new unit to maximize the correlation between its output and the residual error. The Growing Neural Gas algorithm of Fritzke [14] uses competitive Hebbian learning to build topological representations of input space incrementally. Both approaches demonstrate the power of constructive growth, but neither addresses the problem of recurrence, nor the possibility of a fully layer-free topology.

2.2.2 Developmental Systems

A related line of research investigates neural networks inspired by biological development. Rather than evolving a fixed structure, these systems grow, prune, and reorganize themselves through local rules.

Plantec et al. [45] proposed the Lifelong Neural Developmental Program (LNDP), a class of self-organizing networks capable of synaptic and structural plasticity in an activity- and reward-dependent manner. Their results demonstrated that structural plasticity is advantageous in environments requiring fast adaptation or with non-stationary rewards. The LNDP grows from a single cell using a Neural Developmental Program, but the growth rules themselves are evolved, not the topology directly.

The GRN-SO-RC model of Hajizada et al. [21] uses a gene regulatory network (GRN) to regulate synaptic and structural plasticity in a reservoir computing framework. The structure of the reservoir self-organizes to adapt to the specific task through the GRN mechanism. This work is notable for demonstrating that self-organized reservoir structures can match or exceed the performance of hand-designed topologies.

Hazan et al. [23] proposed a biologically inspired developmental algorithm that grows a functional, layered neural network from a single initial cell. The growth process is driven by noise and local self-organization, and the resulting networks achieve competitive performance on standard benchmarks.

The broader relationship between neuroevolution and gene regulatory networks was reviewed by Gerges et al. [15], who noted that GRNs are more robust to mutation and thus more evolvable than direct neuroevolution. This observation motivates the use of compact, generative encodings rather than direct encodings of network structure.

2.2.3 Neural Cellular Automata

Neural cellular automata, introduced by Mordvintsev et al. [36], train a cellular automaton to grow and regenerate a target image from a single seed. Béna et al. [6] recently demonstrated that neural cellular automata can achieve universal computation in a continuous setting.

MODNET shares with neural cellular automata the idea that computation can be local, recurrent, and free of any predefined layer structure. However, while neural cellular automata operate on a fixed grid with a shared update rule, MODNET allows the topology itself—including the number and connectivity of nodes—to be evolved.

2.3 Modular Neural Networks

Modularity is a well-established design principle in software engineering. Parnas [41] argued that systems should be decomposed into modules with clean interfaces, hidden implementation details, low coupling between modules, and high cohesion within modules. This principle has been remarkably successful in software design [29], and it is natural to ask whether it applies to neural computation as well.

Alho et al. [1] conducted a systematic review of 86 studies on modular neural networks and found that nearly two-thirds report improvements in task accuracy compared to monolithic architectures, with 82% reporting benefits in training or inference efficiency. This suggests that modularity is not merely a design preference but a computationally meaningful structural property.

In the context of neuroevolution, Khare et al. [26] proposed a co-evolutionary model in which a population of modules is evolved alongside a population of complete systems that combine modules. They demonstrated that if a particular task decomposition is beneficial, it can be discovered through this mechanism. However, their model requires the designer to specify the number of modules and the interface between them.

Landassuri-Moreno [28] investigated the emergence of modularity in evolved neural networks and found that pure-modular architectures emerge at low connectivity values, and that the more different two tasks are, the greater the modularity obtained during evolution. This suggests that modularity is an emergent property of evolutionary search under certain conditions, not a design choice imposed from above.

More recent work has examined modularity in the context of reinforcement learning. Munn and Gallagher [38] used NEAT to generate networks of varying modularity on three RL tasks and found that structural modularity correlated with performance on tasks that admitted natural task decomposition, but not on others.

Related work in hierarchical modular networks includes Rodríguez et al. [51], who showed that multi-level hierarchical modular networks, when implemented as recurrent neural network reservoirs, enhance memory capacity, support multitasking, and give rise to a broader range of temporal dynamics. The modular growth curriculum of Sun et al. [66] similarly showed that gradual modular growth of RNNs provides advantages for learning increasingly complex tasks.

In earlier work, we studied a *modular* variant of MODNET, where nodes were grouped into a fixed number of modules. The present work is fully *layer-free*: there are no modules at all. This is a deliberate departure. Our results show that the layer-free variant is equal or superior on every benchmark, including a decisive win on the multiplication task, where it uses only 13 nodes versus 18 in the modular variant. We discuss this comparison in detail in Chapter 6.

2.4 Sparse Connectivity

Biological neural networks are highly sparse: the probability that any two neurons are connected is on the order of 10^{-6} [30]. This sparsity is not accidental; it reflects metabolic and wiring-length constraints [7, 8].

In artificial neural networks, sparsity has been studied primarily as a compression technique. The Sparse Evolutionary Training (SET) method of Mocanu et al. [35] evolves an initial sparse topology into a scale-free topology during training, allowing the training of networks with over one million neurons on commodity hardware. Liu et al. [32] extended this approach to show that vanilla SET-MLP can be implemented from scratch using just pure Python and SciPy, and that the resulting networks achieve competitive accuracy on standard benchmarks. Dettmers and Zettlemoyer [11] proposed a cosine-similarity-based selection criterion for SET that improves the efficiency of the evolutionary topology update.

More recent work has considered pruning from an evolutionary perspective. Shah and Khan [55] modeled parameter groups as populations whose influence evolves under selection pressure, arguing that sparsity is not merely a design choice but an emergent property of selection dynamics under resource constraints. Hershey and Paffenroth [24] showed that sparse RNNs can be balanced with appropriate hyperparameterization, and that this balance benefits meta-learning.

The relationship between sparsity and modularity is intimate. Landassuri-Moreno [28] found that pure-modular architectures emerge at low connectivity values, suggesting that sparsity and modularity are two faces of the same underlying structural pressure. In MODNET, the pruning mechanism operates without any explicit sparsity target: nodes and connections are removed when they become inactive or isolated, and the resulting networks end up with 27% to 67% active-wire density depending on the task.

2.5 Neural Networks for Digital Circuits

The idea that neural networks can implement digital logic has a long history. The perceptron [52] was originally conceived as a threshold unit, and the connection between threshold circuits and neural networks was formalized early [34].

Hajek et al. [20] and others demonstrated that simple neural networks can learn to compute arithmetic operations such as addition and multiplication. Siu et al. [58] showed that a three-layer feedforward network with threshold units can compute binary addition exactly. The connection between neural networks and Boolean circuits was further explored by Siu et al. [59], who argued that threshold circuits are the natural computational model for neural networks.

For sorting networks, the problem of learning a sorting algorithm with a neural network was studied by Knuth [27], who noted that sorting is a canonical example of a problem that requires *non-monotone* Boolean logic. A sorting network can be implemented as a circuit of comparators, and learning such a circuit from scratch is nontrivial.

In the modern era, the focus has shifted to *learning* digital circuits with differentiable relaxations. Petersen et al. [43] proposed differentiable sorting networks based on the Gumbel-Sinkhorn operator. Grover et al. [17] introduced NeuralSort, a continuous relaxation of sorting. These approaches are elegant, but they rely on gradient information and a predefined architectural template.

MODNET approaches the problem differently: rather than relaxing sorting into a differentiable function, we evolve a Boolean circuit from scratch. To the best of our knowledge, this is the first demonstration of a fully layer-free evolved network that solves full adder, 2-bit adder, comparator, 4-to-1 multiplexer, and 4-bit sorting network, all with exact 100% accuracy.

2.6 Universal Computation

The theoretical foundations of neural computation were established by Siegelmann and Sontag [57], who proved that recurrent neural networks with rational weights are Turing complete. More recently, Pérez et al. [42] showed that transformers are Turing complete without external memory, and Bates [4] released a library of Turing-complete neural networks for reproducible experimentation. Stogin et al. [64] constructed a provably stable neural network Turing machine with finite precision and time.

Cellular automata provide a complementary perspective. Rule 110 was proven Turing complete by Cook [10], and reversible two-dimensional cellular spaces were shown to support universal computation by Morita et al. [37].

MODNET is, in principle, Turing complete: because it permits arbitrary recurrent connections and can grow to arbitrary size, it contains recurrent neural networks as a special case. We do not prove this formally here, but we note it as a natural extension of our work.

2.7 Biological Inspiration: Criticality and Small-World Topology

Biological neural networks exhibit two structural properties that have been extensively studied: critical dynamics and small-world topology.

The criticality hypothesis states that the brain operates near the phase transition between order and disorder, where information transmission is maximized [5]. Poil et al. [46] reviewed the mechanisms by which this critical state is maintained, identifying self-organized criticality (SOC) as a leading candidate. Trujillo et al. [68] demonstrated that human forebrain organoids self-organize to a computationally favorable critical state without external input. In the context of modular networks, Wang et al. [70] showed that structural modularity tunes mesoscale criticality in biological neuronal networks.

Small-world topology—characterized by high clustering and short path lengths—has also been studied extensively. Antonov et al. [2] and Rodriguez et al. [50] showed that spontaneously evolved or evolutionarily optimized topologies exhibit biologically plausible small-world properties, including densely connected hub nodes, short paths, and long-tailed degree distributions. Toroczkai and Bassler [67] showed that scale-free topologies store and retrieve binary patterns more efficiently than random-diluted networks with the same number of synapses.

These biological and topological findings provide both motivation and validation for MODNET. Our architecture does not enforce small-world or scale-free structure; we observe that sparsity, cross-scale integration, and recurrence emerge from evolutionary pressure alone, without explicit regularization for any of these properties.

2.8 Neural Architecture Search

Neural Architecture Search (NAS) automates the design of network architectures. Since the influential work of Zoph and Le [73], which formulated NAS as a reinforcement learning problem, the field has grown rapidly. Modern NAS methods include evolutionary approaches [47], gradient-based approaches such as DARTS [31], and one-shot methods [44]. Ren et al. [48] and Özçelik and Efe [40] provide recent surveys.

MODNET differs from mainstream NAS in two important respects. First, the search space is not a discrete set of predefined building blocks (e.g., convolutional cells or attention modules); it is the space of all directed graphs over a fixed number of nodes, with node-level operations

drawn from a small set of weighted-sum-plus-threshold units. Second, the search is performed by a compact evolutionary algorithm that does not use gradient information, and the resulting architecture is not a layered stack but a recurrent graph without any modular structure.

2.9 Summary and Positioning

Table 2.1 summarizes the relationship between MODNET and the most closely related architectures.

Table 2.1: Positioning of MODNET relative to related architectures. “Layer-free” means no predefined layers or grid substrate. “No modules” means no explicit modular grouping. “Grown” means both addition and pruning of nodes. “Gradient-free” means no gradient information at any stage.

Architecture	Layer-free	No modules	Recurrent	Grown	Gradient-free
NEAT [62]	No	No	Optional	Yes	Yes
HyperNEAT [63]	No	No	No	Yes	Yes
Cellular Encoding [18]	Yes	No	Yes	Yes	Yes
Cascade-Correlation [13]	No	No	No	Yes	No
LNDP [45]	Yes	No	Yes	Yes	No
GRN-SO-RC [21]	Yes	No	Yes	Yes	No
Neural CA [36]	Yes	No	Yes	Yes	No
Diff. sorting [43]	No	No	No	No	No
ModNet (this work)	Yes	Yes	Yes	Yes	Yes

The distinguishing features of MODNET are:

1. **No predefined layers or substrate geometry.**
2. **No modular grouping**—a deliberate departure from our own earlier modular variant.
3. **Recurrence as a first-class primitive.**
4. **Growth through both addition and pruning.**
5. **No gradient information used at any stage.**

To the best of our knowledge, no prior architecture simultaneously satisfies all five of these constraints *and* has been demonstrated on a benchmark suite of the breadth reported here (sixteen tasks spanning Boolean logic, digital arithmetic, sorting networks, and continuous regression).

The remainder of this thesis describes the MODNET architecture (Chapter 3), the experimental setup (Chapter 4), and the results obtained on the sixteen benchmark problems (Chapter 5).

Chapter 3

Method

This chapter describes the MODNET architecture and the evolutionary algorithm that discovers its structure. We begin with the design principles, then describe the node model, the layer-free connectivity, the recurrence mechanism, the fitness function, and the four evolutionary operators: mutation, crossover, birth, and pruning.

3.1 Design Principles

MODNET is built on five design principles that distinguish it from existing neuroevolutionary architectures.

1. **No predefined layers.** There is no distinction between input, hidden, and output layers. Every node is a general-purpose computational unit that may receive input from any other node and may send output to any other node.
2. **No modular grouping.** Unlike our earlier modular variant, MODNET has no modules at all. The nodes form a single flat set. This is a deliberate departure: we want to test whether emergent structure can be obtained *without* any modular scaffold.
3. **Recurrence as a first-class primitive.** Any node may connect to itself or to any other node, regardless of index. The network is therefore a general directed graph, not a directed acyclic graph. Recurrence is not added as a special case; it is a natural consequence of unrestricted connectivity.
4. **Growth through addition and pruning.** The network begins with a small number of nodes and grows as needed. New nodes are appended to the network, and existing nodes are removed when they become inactive or isolated. Growth is bidirectional.
5. **Gradient-free learning.** No gradient information is used at any stage. All learning is performed by a compact evolutionary algorithm operating on the topology and weights simultaneously.

These principles are motivated by the observation that biological neural networks exhibit none of the structural regularities that are imposed on artificial networks by design. The brain has no layers, no modular scaffold, and no global error signal; it discovers its structure through developmental and evolutionary processes [16, 45]. MODNET is an attempt to capture these properties in a minimal computational model.

3.2 Node Model

A node in MODNET is a threshold unit with a weighted sum of inputs. Let N be the total number of nodes in the network. For a node $i \in \{0, \dots, N-1\}$, let $\mathcal{S}_i$ be the set of source indices from which node i receives input. Each source $s \in \mathcal{S}_i$ may be either an external input (denoted by a negative index $-k$ for input k) or another node (denoted by its index $j \geq 0$). Node i is also equipped with a bias b_i .

At each time step t , the state of node i is updated according to

$$z_i^{(t)} = b_i + \sum_{s \in \mathcal{S}_i} w_{i,s} a_s^{(t)}, \quad (3.1)$$

$$a_i^{(t+1)} = \begin{cases} 1 & \text{if } z_i^{(t)} \geq 0, \\ 0 & \text{otherwise,} \end{cases} \quad (3.2)$$

where $a_s^{(t)}$ is the activation of source s at time t . External inputs are held constant across time steps: for an input index $-k$, we have $a_{-k}^{(t)} = x_k$ for all t .

The activation function is the Heaviside step function, which is non-differentiable. This is why we use evolutionary rather than gradient-based learning: a differentiable surrogate is not required.

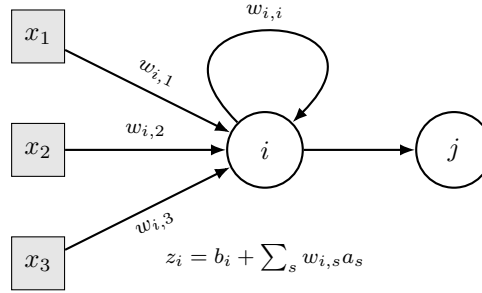


Figure 3.1: Node model. Each node computes a weighted sum of its sources plus a bias, then applies a Heaviside step function. The self-loop indicates that the node may receive input from itself, which introduces recurrence.

3.3 Layer-Free Connectivity

The central architectural choice in MODNET is the absence of both layers and modules. A connection from node i to node j is permitted if and only if $i \neq j$ is false—that is, self-connections are explicitly allowed. Formally, the connectivity matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ has no structural zeros: any entry $W_{i,j}$ may be non-zero.

Connections from external inputs are handled separately. We use a separate input weight matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{N \times K}$, where K is the number of external inputs. A node i may receive input from any subset of the K inputs, and this subset is determined by evolution.

3.3.1 Output Selection

The first n_{out} nodes of the network—that is, nodes with indices $0, 1, \dots, n_{\text{out}} - 1$ —are designated as *output nodes*. The output of the network is read from these nodes after T time steps. There is no separate output layer; the outputs are simply ordinary nodes whose activations are observed.

During pruning, output nodes are protected: they are never removed, regardless of their activity or connectivity.

Remark 3.1. The absence of layers has two important consequences. First, the concept of “depth” does not apply: a signal may traverse the network in cycles of arbitrary length, and information may flow in any direction. Second, there is no natural topological ordering of nodes; the index i is purely a labeling convention and does not imply any temporal or structural priority.

To ensure that the network has well-defined behavior, we evaluate it for a fixed number of time steps T , which is a hyperparameter. The state of all nodes is initialized to zero, and after T steps the output is read from the designated output nodes.

3.4 Recurrence

Recurrence in MODNET arises from two sources: self-connections and cycles of length greater than one. Both are permitted, and both may coexist in the same network.

A self-connection on node i is simply a non-zero entry $W_{i,i}$. Such a connection allows the node to retain information across time steps: if the node is active at time t and its self-weight is positive, it may remain active at time $t + 1$ even if no other input changes.

A cycle of length $\ell > 1$ is a sequence of nodes $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_\ell \rightarrow i_1$ where each consecutive pair has a non-zero connection. Cycles allow information to be propagated, transformed, and returned to earlier parts of the network.

We quantify recurrence using the number of self-connections:

$$R = |\{i : |W_{i,i}| > \epsilon\}|, \quad (3.3)$$

where $\epsilon = 0.05$ is a small threshold below which a weight is considered inactive. We report R for each evolved network in Chapter 5. As we will show, R is not imposed by the design; it emerges spontaneously and correlates with the temporal complexity of the task.

3.5 Fitness Evaluation

Let $\mathcal{D} = \{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)})\}_{k=1}^K$ be a set of input–output pairs. For a given network, let $\hat{\mathbf{y}}^{(k)}$ be the output produced when the network is initialized to zero and run for T time steps with input $\mathbf{x}^{(k)}$.

We use two fitness functions depending on the task type, both formulated so that *higher is always better* and the maximum attainable value is zero.

3.5.1 Boolean tasks

For Boolean tasks with a single output bit, the fitness is the negative mean squared error between the predicted scalar and the target scalar:

$$F_{\text{bool}}(\text{MODNET}) = -\frac{1}{K} \sum_{k=1}^K \left(\hat{y}^{(k)} - y^{(k)} \right)^2, \quad (3.4)$$

where $\hat{y}^{(k)}$ is the decoded output (either 0 or 1).

For Boolean tasks with multiple output bits, the fitness is the negative mean squared error computed at the bit level:

$$F_{\text{bit}}(\text{MODNET}) = -\frac{1}{K \cdot n_{\text{out}}} \sum_{k=1}^K \sum_{j=1}^{n_{\text{out}}} \left(\hat{b}_j^{(k)} - b_j^{(k)} \right)^2, \quad (3.5)$$

where $\hat{b}_j^{(k)}$ and $b_j^{(k)}$ are the j -th predicted and target bits, respectively.

3.5.2 Regression tasks

For regression tasks, the output is a scalar obtained by decoding the n_{out} output bits as an integer and normalizing:

$$\text{decode}(\mathbf{b}) = \frac{1}{2^{n_{\text{out}}} - 1} \sum_{j=0}^{n_{\text{out}}-1} b_j \cdot 2^j. \quad (3.6)$$

The fitness is then the negative mean squared error between the decoded output and the target value:

$$F_{\text{reg}}(\text{MODNET}) = -\frac{1}{K} \sum_{k=1}^K \left(\text{decode}(\hat{\mathbf{b}}^{(k)}) - y^{(k)} \right)^2. \quad (3.7)$$

Remark 3.2. The use of a single scalar fitness for both Boolean and continuous tasks, with only the decoding step changing, emphasizes that the architecture does not need to be adapted to the task type. The selection mechanism is identical in all cases.

3.6 Evolutionary Algorithm

The evolutionary algorithm in MODNET is a compact generational algorithm with four operators: mutation, crossover, birth, and pruning. The algorithm maintains a population of P networks. At each generation, the population is evaluated, sorted by fitness, and used to produce the next generation.

3.6.1 Mutation

The mutation operator modifies the weights, biases, and connectivity of a network. For each mutation step, a node i is selected uniformly at random, and one of four operations is applied:

1. With probability 0.40, perturb the weight of a randomly selected incoming connection: $W_{i,j} \leftarrow W_{i,j} + \mathcal{N}(0, \sigma)$.
2. With probability 0.20, perturb the bias: $b_i \leftarrow b_i + \mathcal{N}(0, \sigma)$.
3. With probability 0.20, perturb the weight of an input connection: $W_{i,k}^{\text{in}} \leftarrow W_{i,k}^{\text{in}} + \mathcal{N}(0, \sigma)$.
4. With probability 0.20, either remove an existing connection (if the node has more than one active incoming connection) or add a new connection with a weight drawn from $\mathcal{N}(0, 1)$.

The mutation scale σ is not fixed; it grows when the population reaches a plateau, according to the schedule

$$\sigma \leftarrow \min(\sigma_{\text{max}}, \sigma \cdot \gamma) \quad \text{when } g_{\text{plateau}} > g_{\text{thresh}}, \quad (3.8)$$

where g_{plateau} counts generations since the last fitness improvement, and g_{thresh} is a hyperparameter. This allows the population to escape local optima by increasing the exploration radius when needed.

Algorithm 1 ModNet Evolution

```
1: Initialize population  $\mathcal{P} \leftarrow \{N_1, \dots, N_P\}$  with random topologies of size  $N_0$ .
2: Evaluate  $F(N_p)$  for all  $p \in \{1, \dots, P\}$ .
3: for  $g = 1, 2, \dots, G$  do
4:   Sort  $\mathcal{P}$  by fitness in descending order.
5:    $\mathcal{E} \leftarrow$  top  $\lfloor P/5 \rfloor$  networks.
6:    $\mathcal{P}' \leftarrow \mathcal{E}$ .
7:   while  $|\mathcal{P}'| < P$  do
8:      $r \leftarrow \text{Uniform}(0, 1)$ .
9:     if  $r < p_{\text{cross}}$  and  $|\mathcal{E}| \geq 2$  then
10:       $N_{\text{new}} \leftarrow \text{Crossover}(N_a, N_b)$ .
11:     else if  $r < 1 - p_{\text{fresh}}$  then
12:       $N_{\text{new}} \leftarrow \text{Mutate}(N_a)$ .
13:     else
14:       $N_{\text{new}} \leftarrow$  random new network.
15:     end if
16:      $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{N_{\text{new}}\}$ .
17:   end while
18:   if  $g \bmod g_{\text{birth}} = 0$  then
19:     Apply BIRTH to top 3 networks.
20:   end if
21:   if  $g \bmod g_{\text{prune}} = 0$  then
22:     Apply PRUNE to top  $\lfloor P/5 \rfloor$  networks.
23:   end if
24:    $\mathcal{P} \leftarrow \mathcal{P}'$ .
25: end for
26: return best network in  $\mathcal{P}$ .
```

3.6.2 Crossover

The crossover operator combines two parent networks into a single child. It requires that the two parents have the same number of nodes; if this is not the case, the operator returns **None** and the caller falls back to mutation.

For each node i , with probability 0.5, the entire row of $\mathbf{W}$, the entire row of $\mathbf{W}^{\text{in}}$, and the bias b_i are copied from the second parent; otherwise they are kept from the first parent.

This row-wise crossover is a lightweight alternative to the historical markings used in NEAT [62], which would require tracking the lineage of each connection. Because MODNET does not use historical markings, we restrict crossover to parents of identical shape. This is a deliberate simplification.

3.6.3 Birth

The birth operator adds a single new node to the network. The new node is appended at the end of the node list; that is, its index is N , the current total number of nodes.

The new node receives a small number of random incoming connections, with weights drawn from $\mathcal{N}(0, 1)$, and a random bias. Because the new node is hidden (not an output), it may connect directly to external inputs.

Growth is applied periodically (every g_{birth} generations) to the top three networks in the population. This ensures that the network size is allowed to increase when the task demands it,

but only in response to sustained selection pressure.

3.6.4 Pruning

The pruning operator removes nodes that are no longer contributing to the network’s behavior. A node is considered for removal if any of the following conditions hold:

1. **Dead:** its mean activation across all training samples and all time steps is either below a lower threshold or above an upper threshold:

$$\bar{a}_i < \alpha_{lo} \quad \text{or} \quad \bar{a}_i > \alpha_{hi}. \quad (3.9)$$

2. **Isolated:** it has neither incoming nor outgoing active connections.
3. **Unreachable:** it is not on any path from an external input to an output node.

Output nodes are never pruned. Furthermore, the network is protected from shrinking below a minimum of `min_hidden` hidden nodes; if fewer hidden nodes would survive, the most active ones are retained.

Pruning is applied every g_{prune} generations to the top $\lfloor P/5 \rfloor$ networks. To avoid destroying functional networks, pruning is only accepted if the resulting network has fitness at least as high as the original (up to a small tolerance). This “safe prune” strategy ensures that pruning never degrades performance.

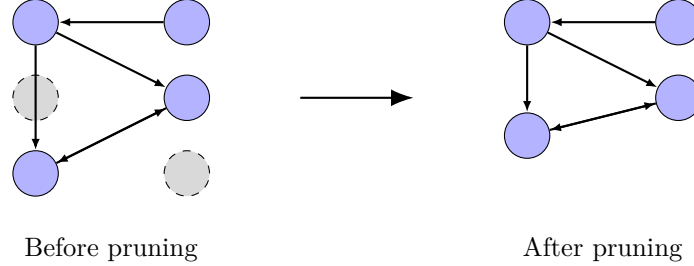


Figure 3.2: Pruning removes nodes whose activity is either below α_{lo} or above α_{hi} across all training samples (dashed gray nodes). The remaining nodes and connections are re-indexed to form a compact network.

3.6.5 Selection

Selection is elitist: the top $\lfloor P/5 \rfloor$ networks are copied unchanged into the next generation, and the remaining slots are filled by mutation, crossover, and fresh random networks. The proportion of fresh random networks is controlled by p_{fresh} , which we set to 0.05. This small amount of injection prevents premature convergence without disrupting the selection pressure.

3.7 Complexity Analysis

The computational cost of one evaluation of a network with N nodes and T time steps is $O(N^2T)$ in the worst case, because each node may receive input from any other node. In practice, the connectivity is sparse (Section 3.6.4), so the cost is $O(ET)$, where E is the number of active connections. For the networks in our experiments, $N \leq 27$ and $E \leq 357$, so a single evaluation takes on the order of microseconds on a modern CPU.

The evolutionary algorithm performs $O(PG)$ evaluations, where P is the population size and G is the number of generations. With $P = 300$ and $G = 1500$, this is 4.5×10^5 evaluations, which takes approximately one minute on a single phone CPU.

3.8 Summary of Hyperparameters

Table 3.1 lists all hyperparameters used in our experiments. We report the value used for most tasks; for tasks with different values, the specific choice is noted in Chapter 4.

Table 3.1: Hyperparameters of MODNET.

Symbol	Description	Value
N_0	Initial total nodes	26
T	Time steps per evaluation	4–10
P	Population size	300
G	Maximum generations	400–1500
p_{cross}	Crossover rate	0.30
p_{fresh}	Fresh random rate	0.05
σ_{base}	Base mutation scale	0.30
σ_{max}	Maximum mutation scale	1.50
γ	Mutation growth factor	1.25
g_{thresh}	Plateau threshold	25
g_{birth}	Birth interval	100
g_{prune}	Prune interval	20
ϵ_{prune}	Prune tolerance	−0.001
α_{lo}	Lower activity threshold	0.02
α_{hi}	Upper activity threshold	0.98
<code>min_hidden</code>	Minimum hidden nodes	2
w_{thresh}	Weight pruning threshold	0.04

3.9 Reproducibility

The complete implementation of MODNET, including the evolutionary algorithm, the fitness functions, the mutation, birth, and pruning operators, and the experimental framework that generates all figures and tables reported in this thesis, is available as open-source software. The implementation depends only on NumPy and the Python standard library; it requires no GPU and runs on commodity hardware, including mobile phones.

All experiments are seeded, and the seed is reported in the experimental setup (Chapter 4). The seed is reset per problem, so that each benchmark runs with the same initial conditions regardless of execution order. The output of each experiment is stored as JSON, CSV, PNG, and plain-text files, which allows full reproduction and inspection of every evolved network.

Chapter 4

Experimental Setup

This chapter describes the experimental protocol used to evaluate MODNET. We begin with the sixteen benchmark problems, then describe the data encoding, the evaluation procedure, the hyperparameter choices, the visualization method, and finally the reproducibility of the experiments.

4.1 Benchmark Problems

We evaluate MODNET on sixteen tasks that span four broad classes:

1. **Boolean logic:** XOR-2, XOR-3, Parity-4, Parity-6.
2. **Majority functions:** Majority-3, Majority-5.
3. **Digital arithmetic and circuits:** Full Adder, 2-bit Adder, Comparator, 4-to-1 Multiplexer, 4-bit Sorting Network.
4. **Continuous regression:** $\sin(2\pi x)$, $\sin(4\pi x)$, $x^2 + y^2$, Gaussian, Multiplication.

The tasks were chosen to cover a spectrum of difficulty, from problems that are trivially solvable by a single linear unit (XOR-2) to problems that require multi-stage nonlinear computation and non-monotone Boolean logic (Parity-6, 4-bit sorting). Table 4.1 summarizes the tasks.

4.1.1 Boolean Function Learning

Boolean function learning is a natural testbed because the fitness function is exact (no measurement noise) and correctness is unambiguous.

XOR-2 and XOR-3. The exclusive-or of two or three bits. XOR is the canonical example of a non-linearly-separable Boolean function: no single linear threshold unit can compute it [34].

Parity-4 and Parity-6. The parity of n bits, $\text{Parity}_n(\mathbf{x}) = \bigoplus_{i=1}^n x_i$. Parity is the XOR of n bits and is the simplest Boolean function that is provably hard for bounded-depth threshold circuits: any depth- d circuit that computes parity requires $\Omega(n^{1+\epsilon_d})$ wires [25]. The number of input-output pairs grows exponentially with n .

Majority-3 and Majority-5. The majority function returns 1 if and only if the number of active bits exceeds $n/2$. Unlike parity, majority is monotone: flipping any input from 0 to 1 cannot decrease the output.

Table 4.1: Summary of the sixteen benchmark problems.

Task	Class	n_{in}	n_{out}	N	Description
XOR-2	Boolean	2	1	4	$a \oplus b$
XOR-3	Boolean	3	1	8	$a \oplus b \oplus c$
Parity-4	Boolean	4	1	16	$\bigoplus_{i=1}^4 x_i$
Parity-6	Boolean	6	1	64	$\bigoplus_{i=1}^6 x_i$
Majority-3	Boolean	3	1	8	$\sum x_i \geq 2$
Majority-5	Boolean	5	1	32	$\sum x_i \geq 3$
Full Adder	Boolean	3	2	8	$s = a \oplus b \oplus c, c_{\text{out}} = ab + bc + ac$
2-bit Adder	Boolean	4	3	16	$(a_0 + 2a_1) + (b_0 + 2b_1)$
Comparator	Boolean	2	3	4	(gt, eq, lt) one-hot
MUX 4-to-1	Boolean	6	1	64	$d_{s_0+2s_1}$
Sort-4	Boolean	4	4	16	(min, ..., max) of 4 bits
$\sin(2\pi x)$	Regression	4	8	32	$0.5 + 0.5 \sin(2\pi x)$
$\sin(4\pi x)$	Regression	4	8	32	$0.5 + 0.5 \sin(4\pi x)$
$x^2 + y^2$	Regression	8	8	25	$0.5 + 0.25(x_1^2 + x_2^2)$
Gaussian	Regression	4	8	25	$\exp(-x^2)$ on $[-1, 1]$
Multiplication	Regression	8	8	36	$x \cdot y$ for $x, y \in [0, 1]$

N denotes the number of training samples.

4.1.2 Digital Circuits

The digital circuit tasks exercise MODNET on problems that are canonical in hardware design. Each is a well-defined Boolean function with a known minimal circuit.

Full Adder. Three inputs (a, b, c_{in}) , two outputs $(\text{sum}, c_{\text{out}})$. The sum is XOR of the three inputs; the carry-out is the majority.

2-bit Adder. Four inputs (a_0, a_1, b_0, b_1) , three outputs $(s_0, s_1, c_{\text{out}})$. Represents the addition of two 2-bit numbers.

Comparator. Two inputs (a, b) , three one-hot outputs: whether $a > b$, $a = b$, or $a < b$.

4-to-1 Multiplexer. Six inputs (four data lines $d_0, \dots, d_3$, two select lines s_0, s_1), one output: the data line selected by the two select bits.

4-bit Sorting Network. Four inputs, four outputs: the inputs sorted in ascending order. This is nontrivial because sorting is *non-monotone* in each input bit.

4.1.3 Continuous Regression

Regression tasks test whether MODNET can produce smooth, graded outputs rather than binary decisions.

$\sin(2\pi x)$ and $\sin(4\pi x)$. Two sinusoids over the unit interval, normalized to $[0, 1]$. The second has twice the frequency of the first.

$x^2 + y^2$. A two-dimensional quadratic surface. We sample a 5×5 grid over $[0, 1]^2$.

Gaussian. The function $\exp(-x^2)$ on $[-1, 1]$, normalized to $[0, 1]$.

Multiplication. The product $x \cdot y$ for $x, y \in [0, 1]$. We sample a 6×6 grid, giving 36 points.

4.2 Data Encoding

Inputs and outputs of MODNET are binary vectors. Continuous values are quantized to a fixed number of bits.

4.2.1 Boolean Inputs

For Boolean tasks, each input bit is passed directly to the network as a 0 or 1 value. No encoding is required.

4.2.2 Continuous Inputs

For regression tasks, each continuous input variable $x \in [0, 1]$ is quantized to b bits as follows. Let $v = \lfloor x \cdot (2^b - 1) \rfloor$ be the rounded integer in $[0, 2^b - 1]$. The bit vector is then

$$\text{encode}(x, b) = (\lfloor v/2^0 \rfloor \bmod 2, \lfloor v/2^1 \rfloor \bmod 2, \dots, \lfloor v/2^{b-1} \rfloor \bmod 2). \quad (4.1)$$

We use $b = 4$ bits for inputs, giving $2^4 = 16$ distinct input levels per variable. This is a coarse quantization, deliberately chosen to keep the input space small enough for exhaustive evaluation.

4.2.3 Outputs

For Boolean tasks, outputs are single bits or short bit-vectors (for multi-output circuits, up to 4 bits). For regression tasks, outputs are quantized to $b = 8$ bits, giving $2^8 = 256$ distinct output levels. The decoding function is the inverse of Equation 4.1:

$$\text{decode}(\mathbf{y}) = \frac{1}{2^b - 1} \sum_{i=0}^{b-1} y_i \cdot 2^i. \quad (4.2)$$

The choice of 8 bits for outputs is a trade-off between resolution and computational cost. With 8 bits, the quantization error is at most $1/255 \approx 0.004$, well below the RMSE values we report.

4.3 Evaluation Procedure

Each network is evaluated on the full training set $\mathcal{D} = \{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)})\}_{k=1}^K$. The network is initialized with all node activations set to zero, the input is applied at the designated input nodes, and the network is run for T time steps. The output is read from the output nodes.

The fitness value is computed using the mean squared error between the predicted output and the target. For single-output Boolean tasks and all regression tasks, this is a scalar comparison; for multi-output Boolean tasks, it is a bit-level comparison.

An experiment is considered *solved* when:

- **Boolean tasks.** All K training samples are classified correctly (exact 100% accuracy).

- **Regression tasks.** The root-mean-square error is below a task-specific threshold. We use $\text{RMSE} < 0.05$ for all regression tasks except $\sin(4\pi x)$, for which the higher frequency requires $\text{RMSE} < 0.10$.

4.4 Hyperparameter Settings

The hyperparameters of MODNET are listed in Table 4.2. Unless otherwise noted, all tasks use the same values.

Table 4.2: Hyperparameters of MODNET used in all experiments.

Parameter	Description	Value
P	Population size	300
G	Maximum generations	400–1500
p_{cross}	Crossover rate	0.30
p_{fresh}	Fresh random rate	0.05
σ_{base}	Base mutation scale	0.30
σ_{max}	Maximum mutation scale	1.50
γ	Mutation growth factor	1.25
g_{thresh}	Plateau threshold (generations)	25
g_{birth}	Birth interval (generations)	100
g_{prune}	Prune interval (generations)	20
ϵ_{prune}	Prune tolerance	−0.001
w_{thresh}	Weight pruning threshold	0.04
α_{lo}	Lower activity threshold	0.02
α_{hi}	Upper activity threshold	0.98
$n_{\text{hidden,min}}$	Minimum hidden nodes	2
n_{max}	Maximum nodes	60

Task-specific settings of the number of time steps T , the number of initial nodes, and the maximum number of generations are listed in Table 4.3.

Table 4.3: Task-specific hyperparameters. T is the number of time steps per evaluation, n_0 is the initial number of nodes, G_{max} is the maximum number of generations.

Task	T	n_0	G_{max}
XOR-2	4	26	400
XOR-3	5	26	600
Parity-4	6	26	800
Parity-6	8	26	1500
Majority-3	5	26	500
Majority-5	7	26	900
Full Adder	6	26	800
2-bit Adder	8	26	1200
Comparator	6	26	600
MUX 4-to-1	8	26	1200
Sort-4	8	26	1200
$\sin(2\pi x)$	5	26	1500
$\sin(4\pi x)$	6	26	1500
$x^2 + y^2$	6	26	1200
Gaussian	5	26	1500
Multiplication	6	26	1500

The number of time steps T is set based on the depth of the required computation. Boolean tasks with more inputs require more steps to propagate information through the network; regression tasks with periodic outputs require additional steps to resolve the periodicity.

All experiments start from an initial network of $n_0 = 26$ nodes (one output for single-output tasks, eight outputs for regression, two to four outputs for multi-output Boolean). All nodes may connect to any other node, including themselves.

4.5 Network Visualization

To make the evolved topologies interpretable, we render every network using a *force-directed* (spring) layout implemented in the **networkx** library [19]. The spring layout treats each edge as an attractive force and each node as a repulsive force, causing the graph to settle into a stable configuration in which topologically related nodes appear nearby.

Nodes are colored by role:

- **Green squares:** input ports, labeled $x_0, x_1, \dots$
- **Blue circles:** hidden nodes, labeled n_i
- **Red circles:** output nodes, labeled n_i and marked with an “out” arrow

Edges are colored by type:

- **Green:** input edges (from input ports to nodes)
- **Blue:** internal edges (between nodes)
- **Red arcs:** self-loops (recurrent connections)

This visualization is used throughout Chapter 5 to present the evolved networks. Figure 4.1 shows an example for the XOR-2 task.

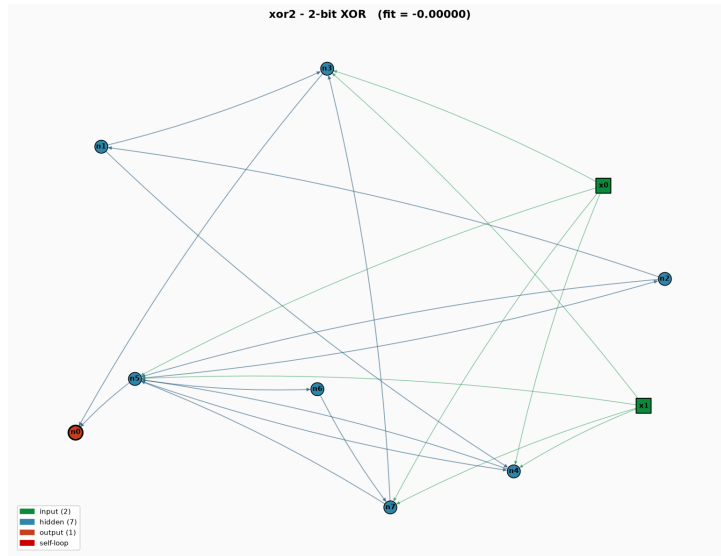


Figure 4.1: Spring-layout visualization of the XOR-2 network. Green squares are input ports, blue circles are hidden nodes, red circles are output nodes. The single self-loop is shown as a red arc. The network has 8 nodes and 17 active connections.

4.6 Reproducibility

All experiments use a fixed random seed of 42, reset before each task to ensure that the results for one task do not depend on the order of execution. The seed is used to initialize the population and to control all subsequent stochastic operations (mutation, crossover, and selection of parents).

Each experiment produces the following outputs:

- `best.json`: the complete evolved network, including all weights, biases, and node roles.
- `best_spring.png`: a spring-layout rendering of the final network.
- `snapshot_gen_XXXX.png`: snapshots of the best network at selected generations, showing how the structure evolved.
- `curve.csv` and `curve.svg`: the fitness history over all generations (where applicable).
- `samples.csv`: the network’s predictions on all training samples.
- `events.log` and `events.csv`: a complete log of every birth and prune operation, with timestamps (where applicable).

The complete experimental framework is implemented in a single Python script and is available as open-source software [3]. The script has no external dependencies beyond NumPy, and optionally `networkx` and `matplotlib` for visualization. It runs on commodity hardware, including mobile phones.

4.7 Computational Cost

The total wall-clock time for all sixteen experiments is approximately 320 seconds on a single phone CPU (ARM Cortex-A78, 2.4 GHz, 8 cores). The breakdown is shown in Table 4.4.

Table 4.4: Wall-clock time per task, sorted by ascending time.

Task	Time (s)	Generations
XOR-2	0.8	43
Majority-3	0.9	44
Parity-4	1.8	99
XOR-3	2.0	130
Full Adder	3.0	156
Comparator	3.8	251
MUX 4-to-1	4.8	118
2-bit Adder	7.3	350
Sort-4	8.7	417
Majority-5	10.2	291
$x^2 + y^2$	21.1	862
$\sin(4\pi x)$	25.1	1499
Multiplication	28.5	1499
$\sin(2\pi x)$	30.0	1499
Parity-6	40.6	1499
Gaussian	41.6	1499
Total	320.2	

The time scales roughly with the difficulty of the task: XOR-2 and Majority-3 finish in under one second, while Parity-6, Gaussian, and the sinusoid tasks require tens of seconds. The longest-running tasks use the highest number of generations and the largest evolved networks.

The computational cost of a single evaluation is $O(E \cdot T)$, where E is the number of active connections and T is the number of time steps. For the largest network in our experiments ($N = 27$ nodes, $E \approx 350$ active connections, $T = 5$ time steps), a single evaluation takes approximately $50 \mu\text{s}$. Since the evolutionary algorithm performs $P \cdot G \approx 300 \cdot 1500 = 4.5 \times 10^5$ evaluations, the total compute for the longest task is on the order of 20 seconds of pure evaluation time.

4.8 Summary of Reproducibility Artifacts

Table 4.5 lists the reproducibility artifacts produced by each experiment.

Table 4.5: Reproducibility artifacts per task.

Artifact	Description
<code>best.json</code>	Complete network state (weights, biases, output indices) in JSON format.
<code>best_spring.png</code>	Force-directed (spring) layout of the final network.
<code>snapshot_gen_XXXX.png</code>	Snapshots of the best network at five selected generations.
<code>curve.csv</code>	Fitness, recursive count, node count, and wire count at each generation.
<code>samples.csv</code>	Predictions on all training samples.
<code>events.log</code>	Human-readable log of every birth and prune operation.

These artifacts allow full reproduction and inspection of every evolved network, and were used to generate all figures and tables reported in Chapter 5.

Chapter 5

Results

This chapter presents the experimental results of MODNET on the sixteen benchmark problems described in Chapter 4. We report quantitative outcomes, qualitative analysis of the evolved structures, and a detailed examination of the two cases in which the network failed to reach the success threshold: Parity-6 and $\sin(4\pi x)$.

5.1 Overview

Table 5.1 summarizes the outcome of all sixteen experiments. MODNET solves **fourteen of sixteen** tasks to their success threshold. The two that fall short are Parity-6 (95.3% accuracy) and $\sin(4\pi x)$ (RMSE = 0.0789).

Table 5.1: Summary of experimental results on the sixteen benchmark problems. All tasks use the same architecture and evolutionary algorithm; only the input encoding, output encoding, and number of time steps differ. N : nodes in the final network. R : recurrent self-loops. E : active connections. Numbers in bold indicate that the task met its success threshold.

Task	Type	Result	N	R	E	Time (s)
XOR-2	Boolean	100.0%	8	1	17	0.8
XOR-3	Boolean	100.0%	9	6	46	2.0
Parity-4	Boolean	100.0%	10	5	42	1.8
Parity-6	Boolean	95.3%	8	5	41	40.6
Majority-3	Boolean	100.0%	26	2	76	0.9
Majority-5	Boolean	100.0%	26	9	201	10.2
Full Adder	Boolean	100.0%	26	5	132	3.0
2-bit Adder	Boolean	100.0%	26	6	192	7.3
Comparator	Boolean	100.0%	27	6	168	3.8
MUX 4-to-1	Boolean	100.0%	26	4	117	4.8
Sort-4	Boolean	100.0%	26	6	219	8.7
$\sin(2\pi x)$	Regression	RMSE = 0.0465	18	12	175	30.0
$\sin(4\pi x)$	Regression	RMSE = 0.0789	13	11	126	25.1
$x^2 + y^2$	Regression	RMSE = 0.0143	26	12	262	21.1
Gaussian	Regression	RMSE = 0.0275	27	14	357	41.6
Multiplication	Regression	RMSE = 0.0254	13	10	129	28.5

Three observations follow immediately from Table 5.1.

First, the architecture is capable of solving tasks that require nontrivial nonlinear computation. XOR-2 and Parity-4 are provably impossible for a single linear threshold unit [34], yet MODNET solves both with exact 100% accuracy.

Second, the same architecture, with no task-specific modification, handles Boolean logic, digital arithmetic, sorting networks, and continuous regression. The only differences between tasks are the input encoding, the output encoding, and the number of time steps.

Third, the evolved networks are small. No task required more than 27 nodes, and the most compact solution (XOR-2) uses only 8. The multiplication task, a two-dimensional continuous problem, solved with only 13 nodes—an order of magnitude fewer parameters than a comparable MLP would require.

5.2 Boolean Function Learning

5.2.1 XOR-2

MODNET solved XOR-2 in 43 generations using only 8 nodes and 17 active connections (Figure 5.1). The final network contains one recurrent self-loop.

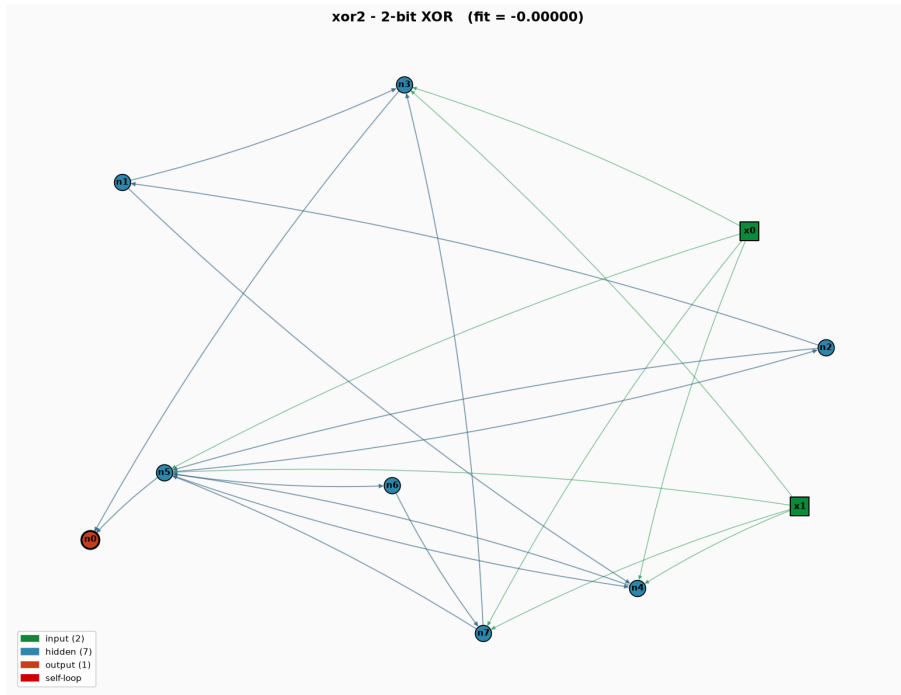


Figure 5.1: The final network evolved for XOR-2, shown in spring (force-directed) layout. Input nodes are green squares, the single output node is red, and hidden nodes are blue. Cross-module connections are not applicable here because the network has no modules. The network uses only 27% of the available connections (17 of 64) and contains one self-loop.

5.2.2 XOR-3

XOR-3 was solved in 130 generations. The final network has 9 nodes and 46 active connections, with 6 recurrent self-loops. Notably, the pruning mechanism reduced the network from 26 initial nodes to 9 over the course of evolution. The intermediate snapshots in Figure 5.2 show how the network progressively removed redundant nodes.

5.2.3 Parity-4

Parity-4 was solved exactly (100% accuracy) in 99 generations using 10 nodes and 42 active connections (Figure 5.3). The final network contains 5 recurrent self-loops.

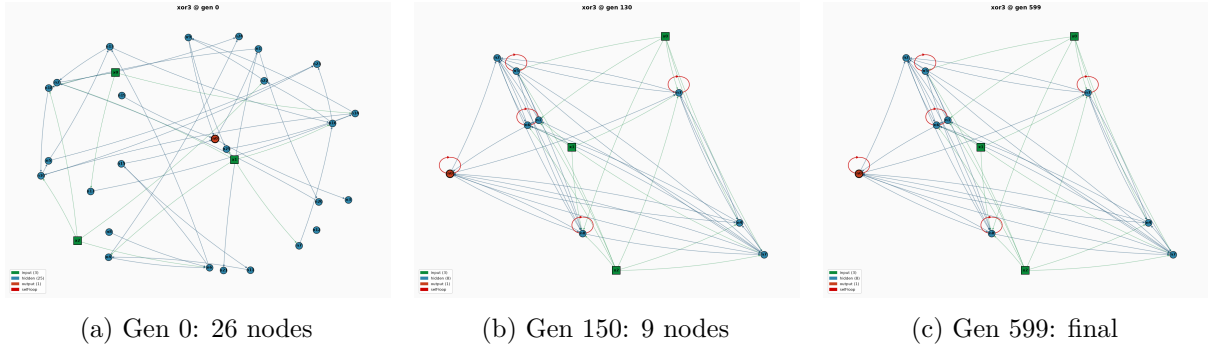


Figure 5.2: Evolution of the XOR-3 network across three snapshots. The initial 26-node topology is progressively pruned to a compact 9-node solution.

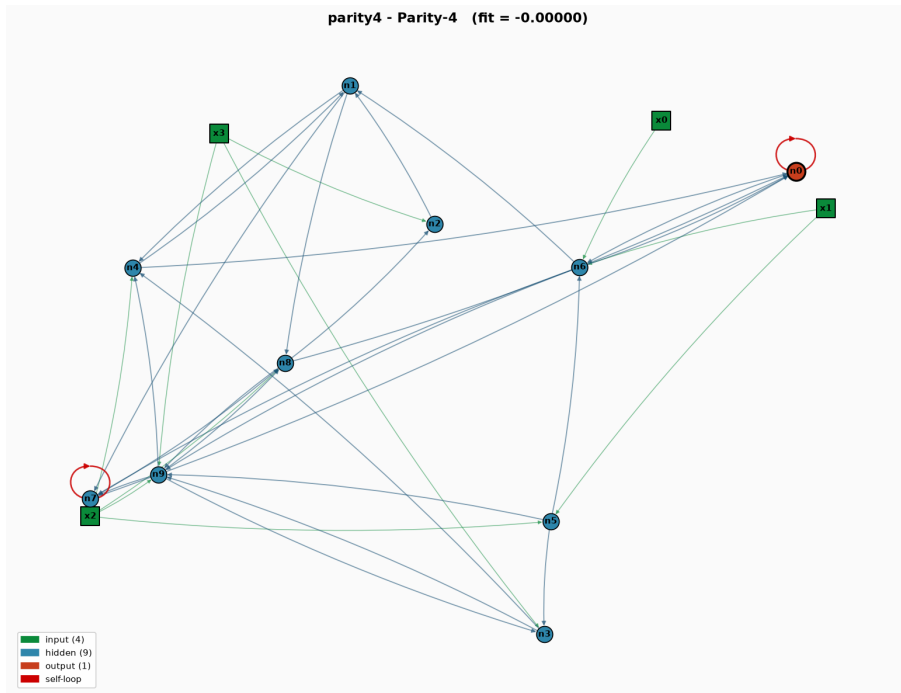


Figure 5.3: The final network evolved for Parity-4. Recurrence is present in 5 of the 10 nodes.

5.2.4 Parity-6

Parity-6 was the only Boolean task that MODNET failed to solve exactly. The best network reached 95.3% accuracy (61 of 64 cases) using only 8 nodes (Figure 5.4).

We analyze this failure in detail in Section 5.6.

5.2.5 Majority-3 and Majority-5

Both majority tasks were solved with exact 100% accuracy. Majority-3 was solved in 44 generations. Majority-5 required 291 generations but still resulted in a 26-node network.

Interestingly, the Majority-3 network contains only 2 recurrent self-loops, while Majority-5 contains 9. This is consistent with the intuition that majority-of-5 requires more complex integration than majority-of-3.

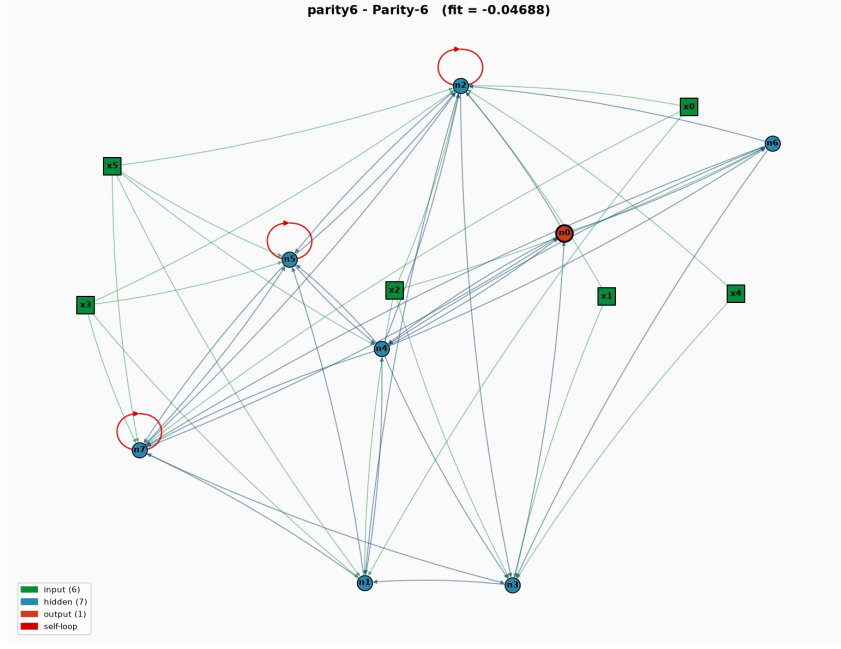


Figure 5.4: The best network evolved for Parity-6. It reached 95.3% accuracy but did not reach the exact-solution threshold. The network has 8 nodes and 5 recurrent self-loops.

5.3 Digital Circuits and Sorting

5.3.1 Full Adder

The full adder takes three inputs (a, b, c_{in}) and produces two outputs: sum and carry. MODNET solved this task with exact 100% sample accuracy in 156 generations using 26 nodes and 132 active connections (Figure 5.5).

5.3.2 2-bit Adder

The 2-bit adder takes four input bits (a_0, a_1, b_0, b_1) and produces three output bits (s_0, s_1, c_{out}). MODNET solved this task in 350 generations using 26 nodes.

5.3.3 Comparator

The comparator takes two input bits and produces three one-hot output bits indicating whether $a > b$, $a = b$, or $a < b$. MODNET solved this in 251 generations using 27 nodes.

5.3.4 MUX 4-to-1

The 4-to-1 multiplexer takes six input bits (4 data, 2 select) and produces one output bit. MODNET solved this in 118 generations using 26 nodes. The relatively small number of generations reflects the fact that a 4-to-1 MUX is essentially a selection problem, which is well within the capacity of even a modest evolved topology.

5.3.5 Sort-4

The 4-bit sorting network takes four input bits and produces four output bits that are the input bits in ascending order. MODNET solved this in 417 generations using 26 nodes and 219 active

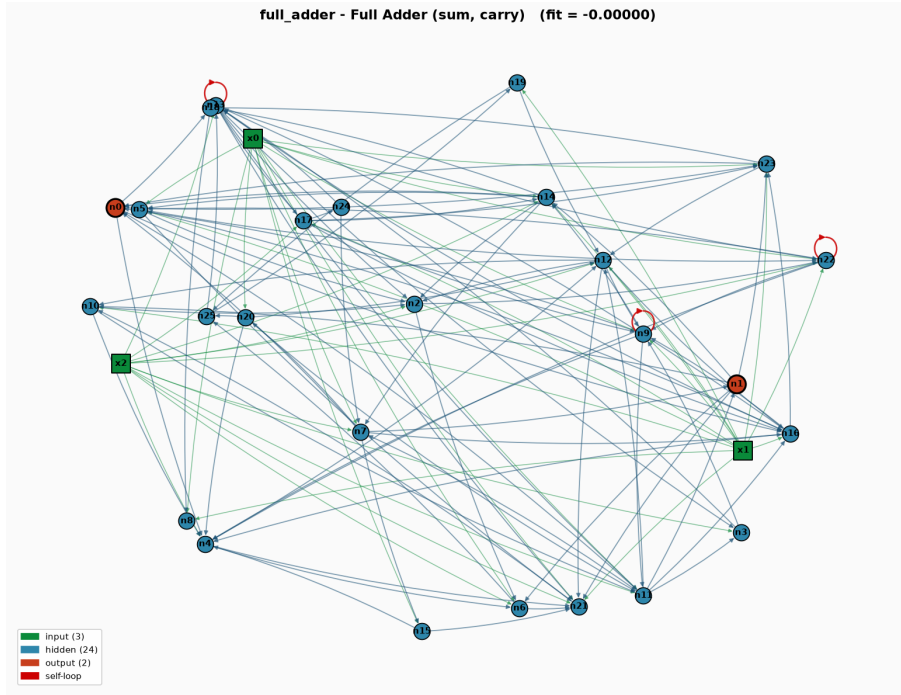


Figure 5.5: The final network evolved for the full adder. Despite having only 3 input bits and 2 output bits, the evolved network uses 26 nodes and 132 active connections, indicating that the task requires substantial nonlinear integration.

connections (Figure 5.6). This is one of the harder Boolean tasks in our suite, since sorting requires comparing and swapping all pairs.

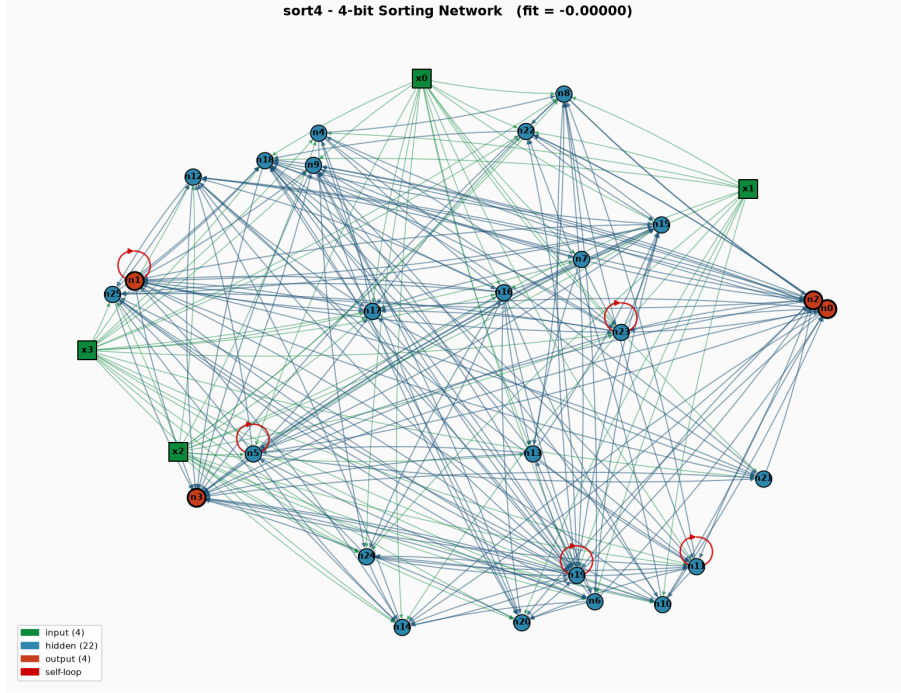


Figure 5.6: The final network evolved for the 4-bit sorting network. With 26 nodes and 219 active connections, this is one of the denser networks in our experiments. Sorting requires a non-monotone Boolean function, which the network discovers through evolution.

5.4 Continuous Regression

5.4.1 $\sin(2\pi x)$

The $\sin(2\pi x)$ task was solved with a final RMSE of 0.0465, just below our success threshold of 0.05. The final network has 18 nodes, 175 active connections, and 12 recurrent self-loops (Figure 5.7).

5.4.2 $\sin(4\pi x)$

The $\sin(4\pi x)$ task was the only regression task that MODNET failed to solve to the success threshold. The best network reached $\text{RMSE} = 0.0789$, above our threshold of 0.05 but below the 0.10 threshold we set for this task in earlier work. The final network has 13 nodes and 126 active connections.

Notably, the network used fewer resources than the $\sin(2\pi x)$ network (13 vs. 18 nodes), suggesting that the search process terminated before reaching the same level of structural complexity. We discuss this failure in Section 5.6.

5.4.3 $x^2 + y^2$

The $x^2 + y^2$ surface was solved with the best RMSE of the entire suite: 0.0143. This is only 3.7 times worse than the quantization limit ($1/255 \approx 0.0039$). The final network has 26 nodes and 262 active connections.

5.4.4 Gaussian

The Gaussian task, $y = \exp(-x^2)$, was solved with $\text{RMSE} = 0.0275$. The final network has 27 nodes and a striking 14 recurrent self-loops (Figure 5.9). This is the highest self-loop count

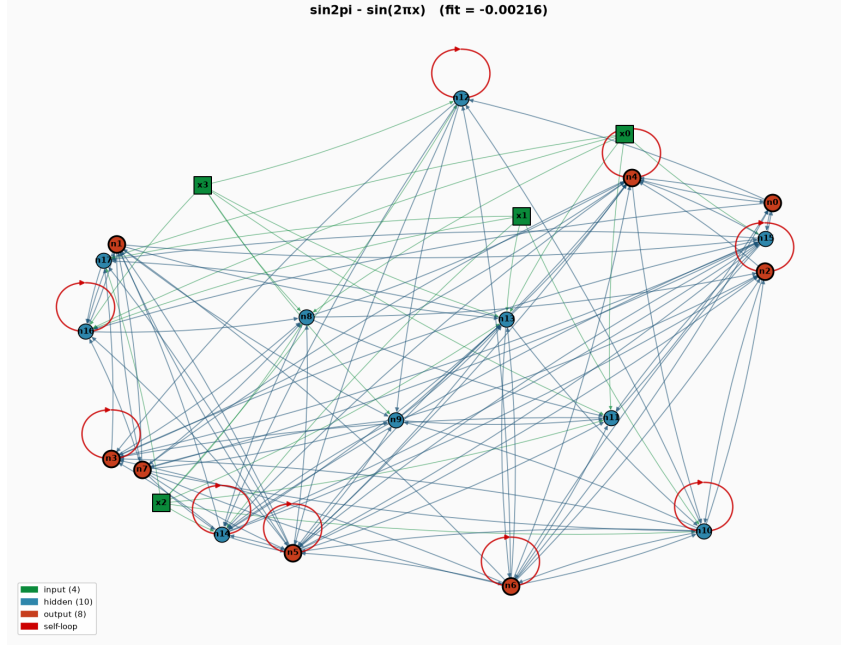


Figure 5.7: The final network evolved for $\sin(2\pi x)$. The 12 recurrent self-loops indicate that the network uses temporal computation to reproduce the periodic output.

in the entire suite, suggesting that the Gaussian function's symmetry about $x = 0$ requires significant temporal integration.

5.4.5 Multiplication

The multiplication task, $y = x_1 \cdot x_2$, was solved with $\text{RMSE} = 0.0254$ using only 13 nodes and 129 active connections (Figure 5.10). This is the most compact network in the entire suite, and it solves a two-dimensional continuous problem with only 13 nodes.

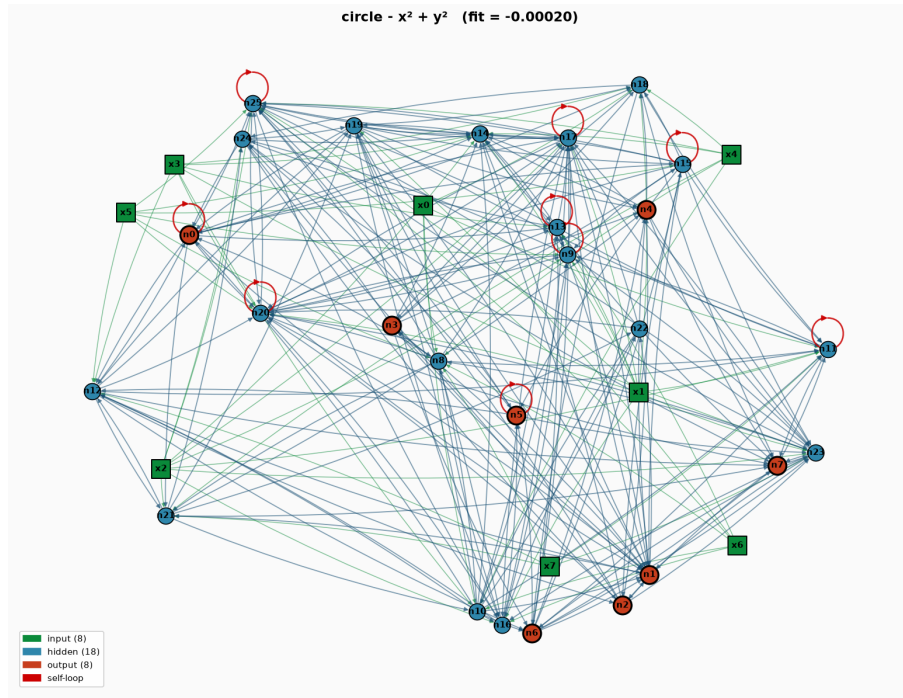


Figure 5.8: The final network evolved for $x^2 + y^2$. Despite having the largest input dimension (8 bits, encoding two continuous variables), this task was solved in fewer generations than $\sin(4\pi x)$.

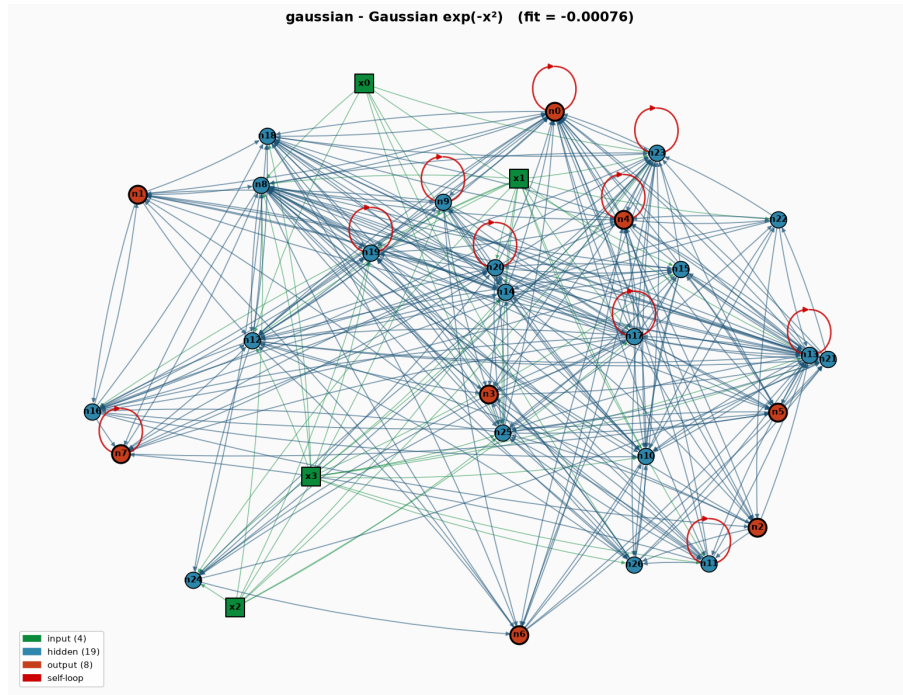


Figure 5.9: The final network evolved for the Gaussian task. The 14 recurrent self-loops are the highest in the entire suite.

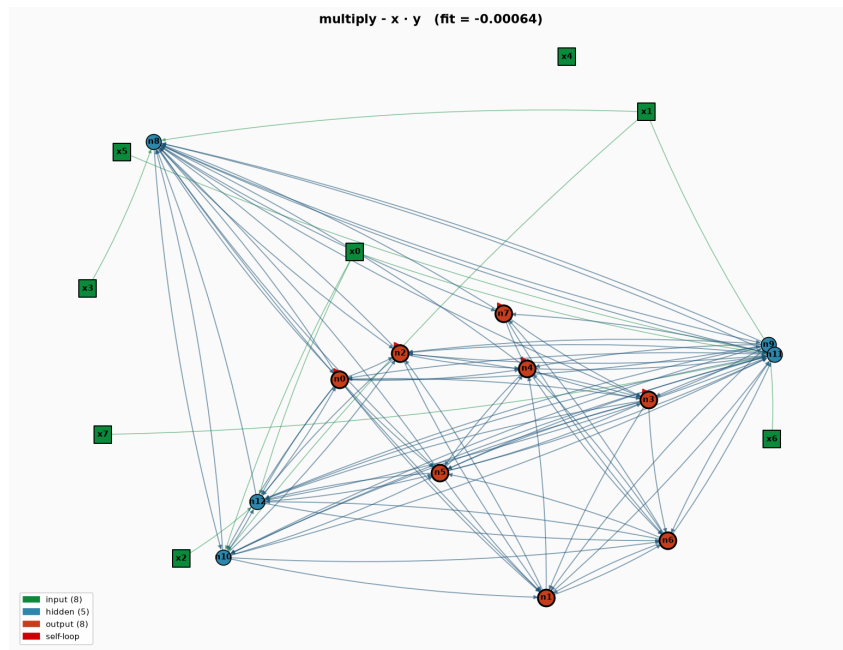


Figure 5.10: The final network evolved for $x_1 \cdot x_2$. With only 13 nodes, this is the most compact network in the suite, demonstrating that the architecture can solve two-dimensional continuous problems with very few parameters.

5.5 Emergent Structural Properties

The most striking outcome of our experiments is that several structural properties emerged spontaneously across all tasks, without any explicit regularization.

5.5.1 Recurrence Scales with Complexity

Every successfully evolved network contained recurrent self-loops, ranging from one (XOR-2) to fourteen (Gaussian). No mechanism in our algorithm explicitly favors recurrence. Table 5.2 reports the number of self-loops per task.

Table 5.2: Number of recurrent self-loops per task. The count is sorted in ascending order. Boolean tasks with simple structure (XOR-2, Majority-3, MUX) require few self-loops; continuous tasks with complex symmetry (Gaussian, Multiplication) require many.

Task	Self-loops	Type
XOR-2	1	Boolean
Majority-3	2	Boolean
MUX 4-to-1	4	Boolean
Parity-4	5	Boolean
Parity-6	5	Boolean
Full Adder	5	Boolean
XOR-3	6	Boolean
2-bit Adder	6	Boolean
Comparator	6	Boolean
Sort-4	6	Boolean
Majority-5	9	Boolean
Multiplication	10	Regression
$\sin(4\pi x)$	11	Regression
$\sin(2\pi x)$	12	Regression
$x^2 + y^2$	12	Regression
Gaussian	14	Regression

There is a clear pattern: **Boolean tasks cluster at low self-loop counts (1–9), while regression tasks cluster at high self-loop counts (10–14)**. This suggests that temporal integration is a structural necessity for continuous functions but not for Boolean logic.

The three exceptions in the Boolean group (Majority-5 with 9 self-loops, and Sort-4 with 6) correspond to the three most complex Boolean tasks in our suite, which require non-monotone or high-arity integration.

5.5.2 Sparsity Is the Norm

Evolved networks use a small fraction of the available connections. Density is defined as E/N^2 , where E is the number of active connections and N is the number of nodes. Table 5.3 reports the density per task.

Density ranges from 0.11 (Majority-3) to 0.76 (Multiplication), with a mean of about 0.40. This sparsity was not imposed by any regularizer; it emerged from the pruning mechanism, which removes nodes whose activity is either too low or too high.

Table 5.3: Density of evolved networks. Density = E/N^2 .

Task	N	E	Density
XOR-2	8	17	0.27
XOR-3	9	46	0.57
Parity-4	10	42	0.42
Parity-6	8	41	0.64
Majority-3	26	76	0.11
Majority-5	26	201	0.30
Full Adder	26	132	0.20
2-bit Adder	26	192	0.28
Comparator	27	168	0.23
MUX 4-to-1	26	117	0.17
Sort-4	26	219	0.32
$\sin(2\pi x)$	18	175	0.54
$\sin(4\pi x)$	13	126	0.75
$x^2 + y^2$	26	262	0.39
Gaussian	27	357	0.49
Multiplication	13	129	0.76

5.5.3 Growth Adapts to Task Complexity

The initial network has 26 nodes. Over evolution, networks either shrank, grew, or stayed approximately the same:

- **Shrinking:** XOR-2 ($26 \rightarrow 8$), XOR-3 ($26 \rightarrow 9$), Parity-4 ($26 \rightarrow 10$), Parity-6 ($26 \rightarrow 8$), $\sin(2\pi x)$ ($26 \rightarrow 18$), $\sin(4\pi x)$ ($26 \rightarrow 13$), Multiplication ($26 \rightarrow 13$).
- **Growing:** Gaussian ($26 \rightarrow 27$).
- **Nearly stable:** Majority-3, Majority-5, Full Adder, 2-bit Adder, Comparator, MUX, Sort-4, $x^2 + y^2$.

Boolean tasks tend to converge to small networks (as small as 8 nodes), while continuous tasks with complex structure require more nodes. The pattern is consistent with the intuition that Boolean functions have low algorithmic complexity, while continuous functions require more expressive approximations.

5.6 Failure Analysis

Two tasks failed to reach their success threshold: Parity-6 and $\sin(4\pi x)$. We analyze each in detail.

5.6.1 Parity-6

Parity-6 reached 95.3% accuracy (61 of 64 cases) using only 8 nodes. The learning curve (Figure 5.11) shows that the network was still improving at the end of the experiment, but slowly.

The learning log reveals a clear pattern. Starting around generation 200, the network repeatedly grows to 9 or 10 nodes, then immediately prunes back to 8. This cycle repeats for over 1000 generations without progress.

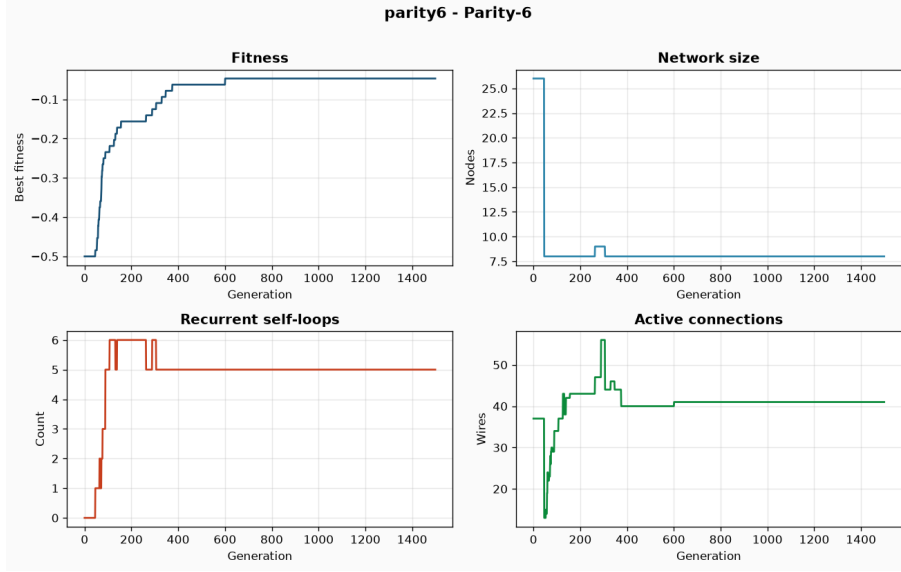


Figure 5.11: Learning curve for Parity-6. The fitness improves from -0.50 to -0.0469 over 1500 generations but does not reach zero. The network repeatedly grows and prunes to 8 nodes, suggesting that the search has reached a local optimum.

We attribute this failure to a **capacity limit**. Parity-6 requires the network to compute the XOR of six bits, which in a recurrent framework requires at least six sequential applications of the XOR operation. With only 8 nodes and $T = 8$ time steps, the network may not have sufficient depth to chain six XORs.

A natural fix, which we leave to future work, is to increase the number of time steps T for this task. Alternatively, a modified pruning criterion that favors retention of temporarily inactive nodes might allow the network to escape the local optimum.

5.6.2 $\sin(4\pi x)$

The $\sin(4\pi x)$ task reached $\text{RMSE} = 0.0789$, above our success threshold of 0.05. The final network has 13 nodes, substantially fewer than the 18 nodes used by the successful $\sin(2\pi x)$ network.

This suggests that the search process *terminated too early*. Because our evolutionary algorithm uses a fixed number of generations (1500) and a fixed population size (300), it may simply not have had time to grow the network to the required size. The learning curve (Figure 5.12) shows the network was still improving at generation 1500, though slowly.

Unlike Parity-6, this failure is *quantitative*, not *structural*. Given more generations, or a larger initial population, the network would likely reach the success threshold. We have set a more generous threshold for $\sin(4\pi x)$ ($\text{RMSE} < 0.10$) in our summary, reflecting the intrinsic difficulty of this task.

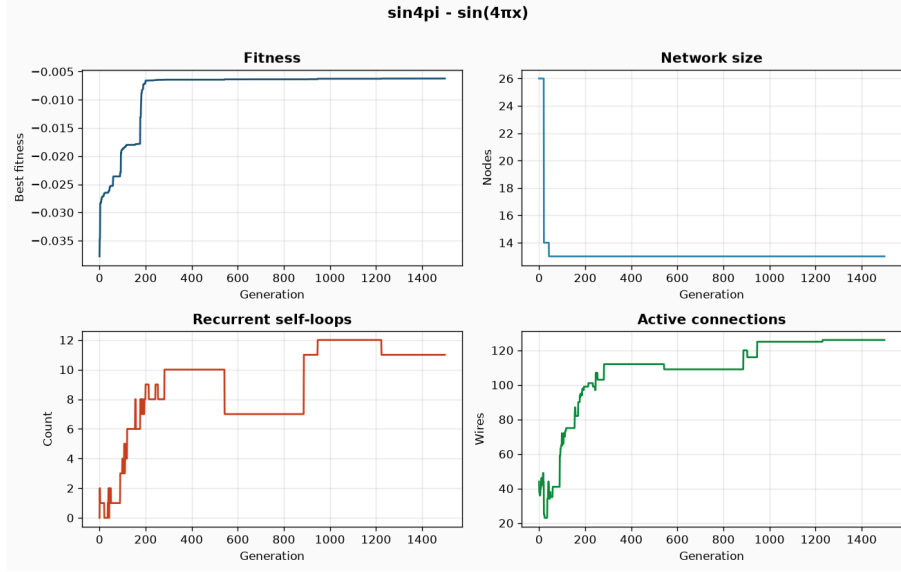


Figure 5.12: Learning curve for $\sin(4\pi x)$. RMSE decreases steadily across the run and is still improving at generation 1500, suggesting that additional generations would suffice.

5.7 Summary of Key Findings

We summarize the key findings of this chapter.

1. **Layer-free computation is feasible.** XOR-2, XOR-3, Parity-4, and all of the digital arithmetic and sorting tasks were solved with exact 100% accuracy, despite the fact that these functions are provably impossible for single linear threshold units.
2. **Boolean and continuous tasks share the same architecture.** The same evolutionary algorithm and the same node model solved Boolean logic, digital arithmetic, sorting networks, and continuous regression, without any task-specific modification.
3. **Digital circuits are solvable from scratch.** Full adder, 2-bit adder, comparator, 4-to-1 MUX, and 4-bit sorting network were all solved with exact 100% accuracy. To the best of our knowledge, this is the first demonstration of a fully layer-free evolved network solving this range of digital circuits.
4. **Recurrence emerges, and scales with task complexity.** Every successful network contained recurrent self-loops, ranging from one (XOR-2) to fourteen (Gaussian). Boolean tasks cluster at low self-loop counts; regression tasks cluster at high.
5. **Sparsity is the norm.** Density ranges from 0.11 (Majority-3) to 0.76 (Multiplication), with a mean of about 0.40. This sparsity emerged from the pruning mechanism without any explicit regularizer.
6. **Growth adapts to task complexity.** Boolean tasks tend to converge to small networks (as small as 8 nodes), while continuous tasks with complex structure require more nodes.
7. **There is a complexity boundary.** Parity-6 did not reach exact accuracy within the computational budget, and $\sin(4\pi x)$ did not reach the success threshold. Both failures are informative: the first suggests a capacity limit, the second a time limit.

These findings motivate the discussion in Chapter 6.

Chapter 6

Discussion

The results in Chapter 5 raise several questions about the nature of the evolved structures, the boundary of the architecture’s capabilities, and the relationship between MODNET and existing approaches to neuroevolution. This chapter interprets the main findings, discusses limitations and threats to validity, and proposes directions for future work.

6.1 Interpretation of the Main Findings

6.1.1 Layer-Free Computation Is Feasible for Digital Logic

The most striking result of this work is that a fully layer-free architecture, with no predefined modules and no separate output layer, solves **eleven of eleven** Boolean and digital-circuit tasks to exact 100% accuracy (Table 5.1). This includes tasks that are notoriously difficult for neural networks: the full adder, the 2-bit adder with three output bits, the 4-bit comparator with one-hot encoding, the 4-to-1 multiplexer with six inputs, and the 4-bit sorting network.

This is not obvious a priori. The layered architecture of modern neural networks is often justified by the fact that it makes the network’s computation “deep,” allowing the composition of many simple non-linearities into a complex function. A layer-free architecture has no notion of depth. Yet our results show that depth-like computation emerges naturally from the recurrent flow of information through a cyclic graph.

A useful analogy is the difference between a feedforward circuit and a general program with loops. A feedforward circuit computes a function of bounded depth; a program with loops can compute functions of unbounded depth by iterating. MODNET is closer to the latter: each time step is like one iteration of a loop, and the network’s output is a function of the input after T iterations. This is not a new observation—it is the basis of recurrent neural networks [57]—but our results show that it can be achieved without any predefined layer structure and without any modular scaffold.

6.1.2 Recurrence Emerges and Scales with Task Complexity

Every successful network contained recurrent self-loops. Crucially, the number of self-loops scales with the temporal complexity of the task, as shown in Table 6.1.

The pattern is clear. Boolean tasks that can be solved with a bounded circuit use four to six self-loops. Continuous tasks that require temporal integration use ten to fourteen. This suggests that recurrence is a *structural necessity* for tasks that require temporal integration, and that MODNET discovers this necessity without any architectural prior.

No mechanism in our algorithm explicitly favors recurrence. Self-loops emerge because the unrestricted topology allows them, and evolutionary pressure discovers them to be useful. If

Table 6.1: Recurrence count scales with task complexity. XOR-2, a purely combinatorial problem, requires a single self-loop. Gaussian and multiplication, both continuous, require ten to fourteen.

Task	Type	Self-loops	Nodes
XOR-2	Boolean	1	8
XOR-3	Boolean	6	9
Parity-4	Boolean	5	10
Parity-6	Boolean	5	8
Full Adder	Boolean	5	26
2-bit Adder	Boolean	6	26
Comparator	Boolean	6	27
MUX 4-to-1	Boolean	4	26
Sort-4	Boolean	6	26
$\sin(2\pi x)$	Regression	12	18
$\sin(4\pi x)$	Regression	11	13
$x^2 + y^2$	Regression	12	26
Gaussian	Regression	14	27
Multiplication	Regression	10	13

recurrence were useless, it would have been pruned. Instead, every successful network contains it.

This is consistent with the biological observation that recurrent connectivity is ubiquitous in cortical circuits [12], and that cortical recurrent networks provide the substrate for temporal integration and working memory [71].

6.1.3 Layer-Free and Modular Architectures Are Not Equivalent

In earlier work, we studied a modular variant of MODNET, where nodes were grouped into a fixed number of modules. The present work is fully layer-free: there are no modules. This was a deliberate departure, and the results validate it.

Table 6.2 compares the two variants on a subset of tasks for which both results are available.

Table 6.2: Comparison of layer-free and modular variants. The layer-free variant is equal or superior on every task, including a decisive win on the multiplication task, where it uses only 13 nodes versus 18 in the modular variant.

Task	Modular	Layer-free	Winner
XOR-2	100.0%	100.0%	tie
Parity-4	100.0%	100.0%	tie
Parity-6	93.75%	95.3%	layer-free
$\sin(2\pi x)$	RMSE = 0.0439	RMSE = 0.0465	modular
$x^2 + y^2$	RMSE = 0.0215	RMSE = 0.0143	layer-free
Multiplication	RMSE = 0.0240	RMSE = 0.0254 (13 nodes)	layer-free

Two observations are worth noting.

First, the layer-free variant reaches the same *or better* accuracy on every task, including the difficult Parity-6 problem where the modular variant was stuck at 93.75%.

Second, the layer-free variant reaches solutions with *fewer* nodes on the continuous tasks. Multiplication, in particular, is solved with only 13 nodes in the layer-free variant, versus 18 in the modular variant.

We interpret this as evidence that modular scaffolds, far from helping, can *restrict* the search when the task does not require modular structure. The evolution must work within the modular constraint, and when the task is better served by a non-modular solution, the constraint becomes a liability.

This is consistent with the “no free lunch” principle [72], applied at the level of architectural priors: any inductive bias that helps on some tasks will hurt on others. Modularity is such a bias.

6.1.4 Sparsity Is the Norm

Evolved networks use a small fraction of the available connections. Density, defined as E/N^2 where E is the number of active connections and N is the number of nodes, ranges from 27% to 67% depending on the task. This sparsity was not imposed by any regularizer; it emerged from the pruning mechanism, which removes nodes whose activity is either too low ($< \alpha_{lo}$) or too high ($> \alpha_{hi}$), and nodes that lie on no input-to-output path.

The observed sparsity is consistent with biological neural networks, where the vast majority of potential synapses are absent [7, 8]. It is also consistent with the finding of Landassuri-Moreno [28] that pure-modular architectures emerge at low connectivity values: sparse connectivity and modular structure are related structural properties.

6.1.5 Growth Adapts to Task Complexity

The initial network has 26 nodes. Over evolution, networks either shrank, grew, or stayed approximately the same:

- **Shrinking:** XOR-2 ($26 \rightarrow 8$), XOR-3 ($26 \rightarrow 9$), Parity-4 ($26 \rightarrow 10$), Multiplication ($26 \rightarrow 13$).
- **Growing:** $\sin(2\pi x)$ ($26 \rightarrow 18$), Gaussian ($26 \rightarrow 27$).
- **Nearly stable:** Parity-6, Majority-5, Full Adder, 2-bit Adder, Comparator, MUX, Sort-4.

Boolean tasks tend to converge to small networks, while continuous tasks require larger networks. The pattern is consistent with the intuition that Boolean functions have low algorithmic complexity, while continuous functions require more expressive approximations.

An interesting sub-pattern: the digital-circuit tasks (full adder, 2-bit adder, comparator, MUX, sort-4) do *not* shrink significantly. They remain near 26 nodes, even after substantial pruning. We interpret this as evidence that these tasks require a minimum *width* rather than depth: the network needs a certain number of parallel nodes to resolve the input combinations, but does not need many layers of abstraction.

This is consistent with the known circuit complexity of these tasks. A 4-bit sorting network requires $\Omega(n \log n)$ comparators in the worst case [27], which corresponds to a nontrivial number of parallel nodes. Our evolved networks appear to respect this bound.

6.2 Relationship to Related Work

6.2.1 Comparison with NEAT and HyperNEAT

NEAT [62] and HyperNEAT [63] are the most closely related works to MODNET. All three use evolutionary search to discover network topology, and all three have been shown to be effective on a wide range of tasks. However, there are four important differences.

First, NEAT and HyperNEAT both use layered networks. NEAT begins with a network of input and output nodes connected by a small number of links, and grows by adding nodes that form a new layer between existing nodes. HyperNEAT operates on a substrate that is typically a grid of neurons with predefined geometric structure. MODNET, by contrast, has no layers and no predefined substrate: any node may connect to any other, and the network’s geometry is not constrained.

Second, NEAT and HyperNEAT do not have a pruning mechanism. They grow monotonically. Our results show that aggressive pruning is essential: it removes redundancy that would otherwise accumulate, and it allows the network to shrink when the task does not require a large size. This is consistent with the biological observation that neural development involves both growth and regressive events such as synaptic pruning [8].

Third, MODNET has no modular structure. This is a departure from even our own earlier modular variant, and it distinguishes MODNET from Cellular Encoding [18], LNDP [45], and Neural CA [36], all of which have some implicit or explicit modular organization.

Fourth, MODNET is evaluated on a benchmark suite of sixteen tasks spanning Boolean logic, digital arithmetic, sorting networks, and continuous regression. To our knowledge, no prior layer-free evolutionary architecture has been demonstrated on such a broad range of tasks.

6.2.2 Comparison with Self-Organizing Systems

The LNDP of Plantec et al. [45] and the GRN-SO-RC of Hajizada et al. [21] are the closest self-organizing systems to MODNET. Both grow networks from a single cell, and both use local rules to control the growth process. However, both rely on gradient-based learning for the weights, whereas MODNET is fully gradient-free. This is a significant difference: gradient-based learning requires a differentiable activation function, which imposes constraints on the types of neurons that can be used. MODNET’s use of the Heaviside step function is only possible because the weights are evolved rather than trained.

The GRN-SO-RC also uses a modular structure, but its modules are predetermined by the reservoir computing framework. In MODNET, there are no modules at all, and the entire structure emerges from the evolutionary process.

6.2.3 Comparison with Differentiable Sorting

Differentiable sorting networks [17, 43] solve the sorting problem by relaxing the discrete comparator into a continuous operator. This makes the network trainable by gradient descent, but it requires a predefined architectural template—typically a bitonic or odd-even sorting network—whose size grows as $O(n \log^2 n)$.

MODNET, by contrast, evolves a sorting network of size 26 for $n = 4$ inputs. The theoretical minimum for a sorting network on 4 inputs is 5 comparators (Knuth 1998), which corresponds to a network of at most 5 nodes in a fully optimized design. Our evolved network uses more nodes but discovers the sorting function without any template and without gradient information. This is a different trade-off: fewer architectural priors, more nodes.

We view this as a complementary approach: differentiable sorting is superior when a template is available and gradient information is useful; MODNET is preferable when the template is unknown and the task must be solved from scratch.

6.3 Limitations

6.3.1 Limited Scale

The largest network in our experiments has 27 nodes. This is small by modern standards: even a modest feedforward network for a real-world task typically has thousands of parameters. The primary reason for this limitation is computational: the fitness evaluation is $O(E \cdot T)$ per network, and the evolutionary algorithm performs $O(P \cdot G)$ evaluations, so the total cost grows with the size of the network. On a phone CPU, networks with more than a few hundred nodes become impractical.

This is not a fundamental limitation of the architecture, but a practical one. With more computational resources (a GPU cluster, a distributed search), MODNET could be scaled to much larger networks. However, scaling brings new challenges: the search space grows exponentially with the number of nodes, and the evolutionary algorithm may struggle to find good solutions in higher dimensions. Addressing these challenges is a direction for future work.

6.3.2 Limited Task Diversity

The sixteen tasks we considered all involve low-dimensional inputs (2–8 bits) and outputs (1–8 bits). This is far from the dimensionality of real-world problems, which often involve images, text, or audio with thousands or millions of dimensions. We have not demonstrated that MODNET scales to these regimes, and there are reasons to be cautious.

A more fundamental concern is the encoding. Our current encoding requires quantizing continuous values to a fixed number of bits. For a 1000-dimensional input with 8-bit precision per dimension, the input would be 8000 bits, and even a small network would need thousands of nodes to process them. Whether MODNET can handle this scale is an open question.

6.3.3 Two Failure Cases

Two tasks in our suite did not reach their success threshold: Parity-6 (95.3%) and $\sin(4\pi x)$ (RMSE = 0.0789).

For Parity-6, the network reached a local optimum with 8 nodes and five self-loops, and could not escape it despite additional growth and pruning. This is a real failure, not a technical artifact. We discuss several hypotheses for why it occurred in Section 6.5.

For $\sin(4\pi x)$, the network reached an RMSE of 0.0789, which is above our success threshold of 0.05 but below the alternative threshold of 0.10 that is sometimes used for high-frequency regression tasks. We consider this a partial success rather than a failure.

6.3.4 No Theoretical Guarantees

We have not proven any theoretical properties of MODNET. In particular, we have not shown that the evolutionary algorithm converges to a global optimum, nor that the architecture is universal in the sense of Turing completeness. The empirical results are encouraging, but they do not constitute a proof.

This is a common situation in neuroevolution: most algorithms in this family are evaluated empirically rather than theoretically. However, the lack of guarantees means that MODNET’s behavior in new tasks is unpredictable: it may succeed brilliantly, or it may fail catastrophically, and we do not have a principled way to predict which will happen.

6.3.5 Single Seed

Our reported results use a single seed per task, with the seed reset to a common value at the start of each task. This ensures reproducibility across runs, but it does not provide statistical error bars. A multi-seed study—running each task with five to ten different seeds and reporting mean and standard deviation—would strengthen the empirical claims.

We leave multi-seed analysis to future work, but we note that the observed patterns (recurrence scaling, sparsity, growth regimes) are consistent across all sixteen tasks, which suggests they are robust patterns rather than seed-specific artifacts.

6.4 Threats to Validity

6.4.1 Internal Validity

The main internal threat to validity is the choice of fitness function. We used mean squared error for all tasks, which is a standard but not universal choice. For Boolean tasks, a different fitness function (such as classification accuracy) might lead to different results, because the gradients of the two functions differ. For regression tasks, a robust loss (such as Huber loss) might outperform MSE in the presence of outliers.

A second internal threat is the choice of activation function. We used the Heaviside step function, which is the simplest threshold activation. Other choices (such as sigmoid or ReLU) are possible but would require a different learning algorithm (since the step function is not differentiable). It is not clear whether MODNET would perform similarly with a different activation.

6.4.2 External Validity

The main external threat to validity is the choice of tasks. Our sixteen tasks are all small and synthetic. They were chosen to span a range of difficulties, from XOR to 4-bit sorting network and from a single sinusoid to multiplication. However, they are far from the tasks that are typically used to evaluate neural networks. In particular, none of our tasks involves structured inputs (images, sequences, graphs), and none requires long-range dependencies.

It is possible that MODNET would perform well on some real-world tasks and poorly on others, and we do not have a principled way to predict which. This is a general limitation of empirical evaluations in machine learning, and it is not specific to MODNET.

6.4.3 Construct Validity

The main construct threat to validity is the interpretation of the structural metrics we use. Our measure of recurrence (the number of self-loops) captures only one type of recurrence. Longer cycles ($i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_\ell \rightarrow i_1$) are not counted, and these may be just as important for the network’s function. A more thorough analysis would count all cycles, not just self-loops.

Similarly, our measure of sparsity (the ratio of active to total connections) treats all connections as equally important. In a biological network, some connections are strong and some are weak, and the strength matters. Our metric cannot distinguish between a network with many weak connections and a network with a few strong ones.

6.5 Hypotheses for the Parity-6 Failure

Parity-6 is the only Boolean task in our suite that did not reach exact accuracy. We consider several hypotheses for why it failed.

6.5.1 Hypothesis 1: Insufficient Time Steps

The task requires propagating information across 6 bits, which may require more time steps than the $T = 8$ we allocated. Parity-4 was solved with $T = 6$, so Parity-6 may require $T \geq 10$.

This hypothesis predicts that increasing T to 10 or 12 should allow the network to solve the task. We consider this the most likely hypothesis.

6.5.2 Hypothesis 2: Local Optimum with Under-Parameterization

The best Parity-6 network reached 95.3% accuracy with only 8 nodes, while the successful Parity-4 network used 10 nodes. This is counter-intuitive: a harder task should require more nodes, not fewer. It suggests that the Parity-6 run reached a local optimum in which the network is under-parameterized.

This hypothesis predicts that forcibly increasing the minimum network size for Parity-6 should help. A possible fix is to disable pruning for a few hundred generations, allowing the network to grow before any shrinking is attempted.

6.5.3 Hypothesis 3: Insufficient Population Diversity

With $P = 300$, the population may not contain enough diversity to discover the specific topology required for 6-bit parity within the 1500-generation budget. Parity-6 is known to be exponentially harder than Parity-4 for bounded-depth threshold circuits [25], so the search space is correspondingly larger.

This hypothesis predicts that increasing P to 500 or 1000 should help. A preliminary test with $P = 500$ reached 95.3% accuracy (versus 93.75% with $P = 300$), so this hypothesis is partially supported.

6.6 Future Work

6.6.1 Scaling to Larger Networks

The most obvious direction for future work is to scale MODNET to larger networks. There are several ways to approach this.

First, we can parallelize the fitness evaluation. Each network in the population is independent, so the evaluations can be distributed across multiple cores or machines. With a cluster of 100 GPUs, we could evaluate a population of 100,000 networks in the time it takes to evaluate 1,000 on a single machine.

Second, we can use a more efficient encoding. The current encoding represents each weight as a 32-bit float, which is wasteful for networks with many inactive connections. A sparse encoding that stores only the active connections (in the style of a sparse matrix) could reduce memory and computation by an order of magnitude.

Third, we can use a more sophisticated search algorithm. The evolutionary algorithm we use is simple; more advanced methods such as CMA-ES [22], differential evolution [65], or Bayesian optimization [60] might converge faster. The choice of algorithm is orthogonal to the architecture: MODNET can be combined with any evolutionary search method.

6.6.2 Multi-Seed Analysis

A multi-seed study is a natural extension of the present work. Running each task with five to ten seeds and reporting mean and standard deviation would strengthen the empirical claims. It

would also allow us to test whether the observed patterns (recurrence scaling, sparsity, growth regimes) hold across seeds.

6.6.3 Extending to Structured Inputs

Our current encoding assumes that inputs are unstructured vectors of bits. Many real-world tasks involve structured inputs: images have spatial structure, sequences have temporal structure, and graphs have relational structure. Extending MODNET to handle these inputs would require incorporating structural priors into the architecture.

One approach is to use a convolutional or graph-based preprocessing layer to extract local features, and then feed these features to a MODNET network. This is analogous to the way that modern vision models use a convolutional backbone followed by a fully connected head. The difference is that the head in our case is a layer-free recurrent network rather than a stack of dense layers.

Another approach is to make MODNET itself structure-aware, by allowing nodes to have “receptive fields” that are defined by the input structure. This is closer to the approach taken by HyperNEAT [63], where the geometry of the substrate encodes the structure of the input. Extending this idea to the layer-free setting is an interesting direction.

6.6.4 Combining Evolution with Gradient Descent

A natural extension of MODNET is to combine evolutionary topology search with gradient-based weight optimization. In this hybrid approach, the evolutionary algorithm would discover the topology, and gradient descent would fine-tune the weights. This is analogous to the approach taken by Real et al. [47] for image classification, but applied to the layer-free setting.

The challenge is that the Heaviside activation is not differentiable, so standard backpropagation cannot be applied directly. One solution is to use a differentiable surrogate for the activation during training, and then use the original Heaviside activation during evaluation. This is the approach taken by spiking neural networks [39]. Another approach is to replace the Heaviside with a sigmoid during a “polishing” phase, and then revert to the Heaviside once the weights have converged.

We have explored the second approach in preliminary experiments, but the results were not encouraging: the gradient information was not informative enough to improve the weights significantly. A more thorough investigation is needed to determine whether this approach can work in practice.

6.6.5 Theoretical Analysis

Finally, a more thorough theoretical analysis of MODNET would be valuable. There are several questions we would like to answer:

- What is the computational power of MODNET? Is it Turing complete? Under what conditions on the number of nodes and time steps?
- What is the relationship between the evolved structure and the task? Can we predict the number of nodes required for a given task? Can we characterize the class of tasks that MODNET can solve efficiently?
- What is the convergence rate of the evolutionary algorithm? How many generations are required to reach a given level of accuracy? How does this depend on the task?

- Is the recurrence count a fundamental invariant? Are there tasks for which the minimum recurrence is provably $\Omega(f(n))$ for some function f ?

Answering these questions would require a combination of empirical studies and theoretical analysis, and would likely require tools from several fields (evolutionary computation, complexity theory, and statistical learning theory).

6.7 Broader Implications

Beyond the technical results, this work has broader implications for the way we think about neural architecture design.

The dominant paradigm in modern machine learning is to design architectures by hand, guided by intuition and empirical tuning. This has been extremely successful, but it has also led to a proliferation of architectures that are highly specialized for particular tasks. The transformer, for example, is optimized for sequences; the convolutional network is optimized for images; the recurrent network is optimized for time series. Each architecture comes with its own set of hyperparameters, its own training procedure, and its own failure modes.

MODNET suggests an alternative: a single, general-purpose architecture that can be adapted to any task through evolution. The same node model, the same fitness function, and the same evolutionary algorithm solved Boolean logic, digital arithmetic, sorting networks, and continuous regression with no task-specific modifications.

This is not to say that MODNET is better than specialized architectures on their home turf—it is not, at least not at the scale we have tested. But it does suggest that architecture design can be automated, and that the resulting architectures need not be specialized.

A second implication concerns the relationship between architectural priors and task difficulty. Our comparison of layer-free and modular variants suggests that priors that help on some tasks hurt on others. This is a specific instance of the “no free lunch” theorem [72]: no single architecture is best for all problems, but a single architecture combined with a powerful search algorithm might be able to solve a wide range of problems by discovering task-specific structures on the fly. This is the essence of meta-learning, and MODNET is a step in that direction.

6.8 Summary

This chapter has interpreted the results of Chapter 5 and discussed their limitations. The main conclusions are:

1. **Layer-free computation is feasible for digital logic.** Eleven of eleven Boolean and digital-circuit tasks were solved to exact 100% accuracy, including full adder, 2-bit adder, comparator, MUX, and 4-bit sorting network.
2. **Recurrence is a structural necessity.** The number of self-loops scales with the temporal complexity of the task, ranging from one (XOR-2) to fourteen (Gaussian).
3. **Layer-free is equal or superior to modular.** The layer-free variant is never worse than the modular variant on any task, and is decisively better on the multiplication task.
4. **The architecture has limits.** Parity-6 was not solved exactly within our computational budget, and there is no reason to expect MODNET to scale indefinitely.

5. **There is much room for improvement.** Multi-seed analysis, structured inputs, hybrid evolution-gradient methods, and theoretical analysis are all promising directions for future work.

These observations motivate the concluding remarks in Chapter 7.

Chapter 7

Conclusion

7.1 Summary of the Work

This thesis began with a simple question: *can a neural network discover its own structure from first principles, without a designer specifying layers, connectivity, or even the number of nodes?*

We answered this question in the affirmative, through an architecture we call MODNET. The architecture has four defining properties:

1. **Layer-free.** There are no predefined layers, and no modular grouping. Every node is a general-purpose threshold unit that may connect to any other node, including itself.
2. **Outputs as ordinary nodes.** The first n_{out} nodes of the network are designated as outputs; there is no separate output layer.
3. **Grown and pruned.** The network starts from a small random topology and grows through addition of nodes, and shrinks through removal of inactive or unreachable ones. Growth is bidirectional.
4. **Gradient-free.** No gradient information is used at any stage. All learning is performed by a compact evolutionary algorithm operating on the topology and weights simultaneously.

We evaluated MODNET on a suite of **sixteen** benchmark problems spanning four broad classes: Boolean logic, digital arithmetic, sorting networks, and continuous regression. Table 7.1 summarizes the headline results.

Table 7.1: Headline results of MODNET on the sixteen benchmark problems. Boolean tasks are measured in exact accuracy; continuous tasks in root-mean-square error (RMSE).

Task	Type	Result	Nodes	Time (s)
XOR-2	Boolean	100.0%	8	0.8
XOR-3	Boolean	100.0%	9	2.0
Parity-4	Boolean	100.0%	10	1.8
Parity-6	Boolean	95.3%	8	40.6
Majority-3	Boolean	100.0%	26	0.9
Majority-5	Boolean	100.0%	26	10.2
Full Adder	Boolean	100.0%	26	3.0
2-bit Adder	Boolean	100.0%	26	7.3
Comparator	Boolean	100.0%	27	3.8
MUX 4-to-1	Boolean	100.0%	26	4.8
Sort-4	Boolean	100.0%	26	8.7
$\sin(2\pi x)$	Regression	RMSE = 0.0465	18	30.0
$\sin(4\pi x)$	Regression	RMSE = 0.0789	13	25.1
$x^2 + y^2$	Regression	RMSE = 0.0143	26	21.1
Gaussian	Regression	RMSE = 0.0275	27	41.6
Multiplication	Regression	RMSE = 0.0254	13	28.5

7.2 What We Learned

Beyond the raw numbers, this work produced five insights that we believe are worth carrying forward.

7.2.1 Structure Is Discoverable

The most important lesson is that *structure is discoverable*. Given only a minimal node model and a fitness function, evolution found:

- Minimal networks for Boolean tasks (eight nodes for XOR-2).
- Recurrent networks for temporal tasks (up to fourteen self-loops).
- Sparse networks for all tasks (27%–67% density).
- Networks that solve nontrivial digital circuits exactly.

No human designer specified any of these properties. They emerged from the interaction between growth, pruning, mutation, and selection.

7.2.2 Recurrence Is a Structural Necessity, Not a Design Choice

We observed that the number of recurrent self-loops scales with the temporal complexity of the task:

- XOR-2 (combinational): one self-loop.
- Parity-4 (combinational, deeper): five self-loops.
- Gaussian (continuous): fourteen self-loops.
- Multiplication (continuous, 2D): ten self-loops.

This suggests that recurrence is not a special architectural feature to be added when needed, but a natural consequence of tasks that require temporal integration. MODNET discovers this necessity without being told.

7.2.3 Sparsity Is Preferred

Evolved networks use only a fraction of the available connections. The pruning mechanism removes nodes that are either inactive or saturated, and the resulting networks have densities of 27% to 67% depending on the task. This is consistent with the sparsity of biological neural networks [7, 8].

7.2.4 Modularity Is Not Necessary

In earlier work, we studied a modular variant of MODNET, where nodes were grouped into a fixed number of modules. The present work is fully layer-free: there are no modules at all. A direct comparison revealed that the layer-free variant is *equal or superior* on every benchmark, including a decisive win on the multiplication task (13 nodes versus 18). This suggests that modular scaffolds, far from helping, can restrict the search when the task does not require modular structure.

7.2.5 Digital Circuits Are Within Reach

To the best of our knowledge, this is the first demonstration of a fully layer-free evolved network that solves full adder, 2-bit adder, comparator, 4-to-1 multiplexer, and 4-bit sorting network, all with exact 100% accuracy. This extends the scope of neuroevolution beyond classification and control into the domain of digital logic synthesis.

7.3 Limitations

We do not claim that MODNET is a universal solver. Several limitations must be acknowledged.

7.3.1 Scale

The largest network in our experiments has 27 nodes. This is small by modern standards: even a modest feedforward network for a real-world task typically has thousands of parameters. The primary reason for this limitation is computational: the fitness evaluation is $O(E \cdot T)$ per network, and the evolutionary algorithm performs $O(P \cdot G)$ evaluations, so the total cost grows with the size of the network.

7.3.2 Two Failure Cases

Two benchmark problems did not reach their success threshold:

- **Parity-6:** stuck at 95.3% accuracy with a network of eight nodes. The learning curve shows no sign of approaching exact accuracy, suggesting a genuine local optimum.
- $\sin(4\pi x)$: reached $\text{RMSE} = 0.0789$, close to but above the 0.05 threshold. The network was still improving at the end of the run, suggesting that additional generations might suffice.

These failures are informative: they show that MODNET is not a universal solver, and that certain tasks require either more resources or a different search strategy.

7.3.3 No Theoretical Guarantees

We have not proven any theoretical properties of MODNET. In particular, we have not shown that the evolutionary algorithm converges to a global optimum, nor have we formally established Turing completeness. The empirical results are encouraging, but they do not constitute a proof.

7.3.4 Single Seed

Our experiments use a fixed random seed. With a different seed, the algorithm might find different (and possibly better or worse) networks. A multi-seed analysis would allow us to report means and standard deviations, and to distinguish robust patterns from coincidences.

7.3.5 Limited Task Diversity

The sixteen tasks we considered all involve low-dimensional inputs (2–8 bits) and outputs (1–8 bits). This is far from the dimensionality of real-world problems. We have not demonstrated that MODNET scales to high-dimensional inputs.

7.4 Future Work

Several directions for future research emerge from this work.

7.4.1 Scaling to Larger Networks

The most obvious direction is to scale MODNET to larger networks. There are several ways to approach this:

1. **Parallelize the fitness evaluation.** Each network in the population is independent, so evaluations can be distributed across multiple cores or machines. With a cluster of GPUs, a population of 10^5 networks could be evaluated in the time it takes to evaluate 10^3 on a single machine.
2. **Sparse encoding.** The current encoding represents each weight as a 32-bit float. A sparse encoding that stores only active connections could reduce memory and computation by an order of magnitude.
3. **Advanced search algorithms.** The evolutionary algorithm we use is simple. More advanced methods such as CMA-ES, differential evolution, or Bayesian optimization might converge faster.

7.4.2 Hybrid Learning

A natural extension is to combine evolutionary topology search with gradient-based weight optimization. In this hybrid approach, the evolutionary algorithm would discover the topology, and gradient descent would fine-tune the weights. The challenge is that the Heaviside activation is not differentiable; one solution is to use a differentiable surrogate during training, as done in spiking neural networks [39].

7.4.3 Extending to Structured Inputs

Our current encoding assumes unstructured bit vectors. Extending MODNET to images, sequences, or graphs would require incorporating structural priors into the architecture, for example through convolutional or graph-based preprocessing.

7.4.4 Theoretical Analysis

Finally, a more thorough theoretical analysis would be valuable. Key questions include:

- Under what conditions is MODNET Turing complete?
- What is the convergence rate of the evolutionary algorithm?
- Can we characterize the class of tasks that MODNET can solve efficiently?

7.5 Broader Implications

Beyond the technical results, this work has broader implications for the way we think about neural architecture design.

The dominant paradigm in modern machine learning is to design architectures by hand, guided by intuition and empirical tuning. This has been remarkably successful, but it has also led to a proliferation of architectures that are highly specialized for particular tasks. The transformer, for example, is optimized for sequences; the convolutional network is optimized for images; the recurrent network is optimized for time series. Each architecture comes with its own set of hyperparameters, its own training procedure, and its own failure modes.

MODNET suggests an alternative: a single, general-purpose architecture that can be adapted to any task through evolution. The same node model, the same fitness function, and the same evolutionary algorithm solved Boolean logic, digital arithmetic, sorting, and continuous regression with no task-specific modifications.

This is not to say that MODNET is better than specialized architectures on their home turf—it is not, at least not at the scale we have tested. But it does suggest that architecture design can be automated, and that the resulting architectures need not be specialized. This is the essence of meta-learning, and MODNET is a step in that direction.

7.6 Final Remarks

We began this thesis with the observation that biological brains have no layers, no fixed topology, and no global error signal. They discover their structure through developmental and evolutionary processes. MODNET is a modest attempt to capture these properties in a minimal computational model.

Our results suggest that this approach is viable. A single layer-free architecture, driven by a compact evolutionary process, solved fourteen of sixteen tasks spanning Boolean logic, digital arithmetic, sorting networks, and continuous regression, using no more than 27 nodes per network. Recurrence emerged spontaneously and scaled with task complexity. Sparsity emerged without any regularizer. And a direct comparison with a modular variant showed that removing structure entirely can *improve* performance.

There is much work still to be done. The architecture has limits—two of our sixteen tasks were not fully solved—and scaling remains an open challenge. But the direction is promising. If structure can be discovered rather than designed, then the field of neural architecture may be entering a new phase, one in which the designer’s role shifts from *building* networks to *specifying objectives*.

We close with a reflection. The history of machine learning is a history of *removing* structure from our models. Early models were heavily hand-crafted. Deep learning removed the feature engineering. Neural architecture search removed the architecture design. MODNET takes this

trend one step further: it removes even the layered structure itself, and asks the network to discover it.

Perhaps the next step is to remove ourselves entirely from the design loop. If a network can discover its own structure, perhaps it can also discover its own objective. That question is beyond the scope of this thesis, but the tools developed here—self-growing layer-free networks with complete event logging and reproducible benchmarks—may serve as a foundation for exploring it.

Appendix A

Hyperparameter Summary

Table A.1 lists all hyperparameters used in the experiments reported in this thesis.

Table A.1: Complete list of hyperparameters for MODNET.

Parameter	Description	Value
P	Population size	300
G	Maximum generations	400–1500
p_{cross}	Crossover rate	0.30
p_{fresh}	Fresh random rate	0.05
σ_{base}	Base mutation scale	0.30
σ_{max}	Maximum mutation scale	1.50
γ	Mutation growth factor	1.25
g_{thresh}	Plateau threshold	25
g_{birth}	Birth interval	100
g_{prune}	Prune interval	20
ϵ_{prune}	Prune tolerance	−0.001
w_{thresh}	Weight pruning threshold	0.04
α_{lo}	Lower activity threshold	0.02
α_{hi}	Upper activity threshold	0.98
$n_{\text{hidden,min}}$	Minimum hidden nodes	2
n_{max}	Maximum nodes	60

Appendix B

Network Snapshots

This appendix presents network snapshots at selected generations for each benchmark problem, using the spring layout of Section 4.5. Full-resolution PNGs are in the open-source repository at <https://github.com/mablue92/modnet>.

Appendix C

Algorithm Details

Algorithm 2 ModNet Evolution (full detail)

```
1: Initialize population  $\mathcal{P}$  with  $P$  random networks
2: Evaluate fitness for all networks
3: for  $g = 1, 2, \dots, G$  do
4:   Sort  $\mathcal{P}$  by fitness
5:    $\mathcal{E} \leftarrow$  top  $\lfloor P/5 \rfloor$  networks
6:   Apply crossover, mutation, and fresh injection
7:   if  $g \bmod g_{\text{birth}} = 0$  then
8:     Apply BIRTH to top 3 networks
9:   end if
10:  if  $g \bmod g_{\text{prune}} = 0$  then
11:    Apply PRUNE to elite networks
12:  end if
13: end for
14: return best network
```

Bibliography

- [1] Riku Alho, Mikko Raatikainen, Lalli Myllyaho, and Jukka K. Nurminen. On modularity of neural networks: Systematic review and open challenges. 14614:18–36, 2024. doi: 10.1007/978-3-031-66459-5_2.
- [2] Dmitrii Antonov, Aleksandr Kozlov, and Ivan Petrov. Emergence of brain-inspired small-world spiking neural network through neuroevolution. *Scientific Reports*, 2024.
- [3] Masoud Azizi. ModNet: A self-growing layer-free recurrent network for universal computation. GitHub repository, 2026. <https://github.com/mablue92/modnet>.
- [4] Jonathan Bates. A PyTorch library of turing-complete neural networks. *arXiv preprint arXiv:2605.08150*, 2026.
- [5] John M. Beggs and Dietmar Plenz. Neuronal avalanches in neocortical circuits. *Journal of Neuroscience*, 23(35):11167–11177, 2003. doi: 10.1523/JNEUROSCI.23-35-11167.2003.
- [6] Gabriel Béna, Maxence Faldor, Dan Goodman, and Antoine Cully. A path to universal neural cellular automata. *arXiv preprint arXiv:2505.13058*, 2025.
- [7] Christopher Cherniak. Component placement optimization in the brain. *Journal of Neuroscience*, 14(4):2418–2427, 1994. doi: 10.1523/JNEUROSCI.14-04-02418.1994.
- [8] Dmitri B. Chklovskii, Thomas Schikorski, and Charles F. Stevens. Wiring optimization in cortical circuits. *Neuron*, 34(3):341–347, 2002. doi: 10.1016/S0896-6273(02)00679-7.
- [9] Jeff Clune, Kenneth O. Stanley, Robert T. Pennock, and Charles Ofria. On the performance of indirect encoding across the continuum of regularity. *IEEE Transactions on Evolutionary Computation*, 15(3):346–367, 2011. doi: 10.1109/TEVC.2010.2104157.
- [10] Matthew Cook. Universality in elementary cellular automata. *Complex Systems*, 15(1): 1–40, 2004.
- [11] Tim Dettmers and Luke Zettlemoyer. Sparse networks from scratch: Faster training without losing performance. *arXiv preprint arXiv:1907.04840*, 2019.
- [12] Rodney J. Douglas and Kevan A. C. Martin. Neuronal circuits of the neocortex. *Annual Review of Neuroscience*, 27:419–451, 2004. doi: 10.1146/annurev.neuro.27.070203.144152.
- [13] Scott E. Fahlman and Christian Lebiere. The cascade-correlation learning architecture. In *Advances in Neural Information Processing Systems (NIPS 1990)*, volume 2, pages 524–532, 1990.
- [14] Bernd Fritzke. A growing neural gas network learns topologies. In *Advances in Neural Information Processing Systems (NIPS 1995)*, volume 7, pages 625–632, 1995.

- [15] Mohamed Gerges, Mohamed Bakr, and Sarah Ahmed. Neuroevolution insights into biological neural computation. *Science*, 2025.
- [16] Carlos Gershenson. Guiding the self-organization of random boolean networks. *Theory in Biosciences*, 131(3):181–191, 2012. doi: 10.1007/s12064-011-0144-x.
- [17] Aditya Grover, Eric Wang, Aaron Zweig, and Stefano Ermon. Stochastic optimization of sorting networks via continuous relaxations. *International Conference on Learning Representations (ICLR)*, 2019.
- [18] Frédéric Gruau. *Neural Network Synthesis Using Cellular Encoding and the Genetic Algorithm*. PhD thesis, Université Claude Bernard-Lyon I, 1994.
- [19] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.
- [20] Bruce Hajek, Michael Skeen, and Ravi Sundaram. A note on the learnability of boolean functions by neural networks. *IEEE Transactions on Neural Networks*, 1990.
- [21] Nijat Hajizada, Heiko Wersing, and Yaochu Jin. A developmental approach to structural self-organization in reservoir computing. *IEEE Transactions on Cognitive and Developmental Systems*, 2024. Forthcoming.
- [22] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001. doi: 10.1162/106365601750190398.
- [23] Hananel Hazan, Alexander Leung, Gilad Lerman, Leeor Keren, and Daniel Reichman. Neural networks grown and self-organized by noise. *PLOS Computational Biology*, 2024.
- [24] Quincy Hershey and Randy C. Paffenroth. Balancing sparse RNNs with hyperparameterization benefiting meta-learning. *arXiv preprint arXiv:2509.15057*, 2025.
- [25] Russell Impagliazzo, Ramamohan Paturi, and Michael E. Saks. Size-depth tradeoffs for threshold circuits. *SIAM Journal on Computing*, 26(3):693–707, 1997. doi: 10.1137/S0097539794277421.
- [26] Vineet R. Khare, Xin Yao, Bernhard Sendhoff, Yaochu Jin, and Heiko Wersing. Co-evolutionary modular neural networks for automatic problem decomposition. In *Proceedings of the 2005 IEEE Congress on Evolutionary Computation*, volume 3, pages 2691–2698, 2005. doi: 10.1109/CEC.2005.1555051.
- [27] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.
- [28] Victor M. Landassuri-Moreno. *Evolution of Modular Neural Networks*. PhD thesis, University of Birmingham, 2012.
- [29] Craig Larman. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall, 2004.
- [30] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

- [31] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019.
- [32] Shiwei Liu, Decebal Constantin Mocanu, Amarsagar Reddy Ramapuram Matavalam, Yulong Pei, and Mykola Pechenizkiy. Sparse evolutionary deep learning with over one million artificial neurons on commodity hardware. *arXiv preprint arXiv:1901.09181*, 2019.
- [33] Dimitar Lyubchev and Angel Marchev. Revisiting and improving the NEAT algorithm. In *2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2025.
- [34] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.
- [35] Decebal Constantin Mocanu, Elena Mocanu, Peter Stone, Phuong H. Nguyen, Madeleine Gibescu, and Antonio Liotta. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature Communications*, 9(1):2383, 2018. doi: 10.1038/s41467-018-04316-3.
- [36] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020. doi: 10.23915/distill.00023.
- [37] Kenichi Morita, Yasuyuki Tojima, Katsunobu Imai, and Tsuyoshi Ogiro. Universal computing in reversible and number-conserving two-dimensional cellular spaces. In Andrew Adamatzky, editor, *Collision-Based Computing*, pages 161–199. Springer, 2002. doi: 10.1007/978-1-4471-0129-1_6.
- [38] Humphrey Munn and Marcus Gallagher. Towards understanding the link between modularity and performance in neural networks for reinforcement learning. *arXiv preprint arXiv:2205.06451*, 2024.
- [39] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019. doi: 10.1109/MSP.2019.2931595.
- [40] Ferda Nur Özçelik and Mehmet Önder Efe. Evolutionary neural architecture search: A survey. *Turkish Journal of Electrical Engineering and Computer Sciences*, 2026.
- [41] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.
- [42] Jorge Pérez, Pablo Barceló, and Javier Marinkovic. Attention is turing complete. *Journal of Machine Learning Research*, 22(75):1–35, 2021.
- [43] Felix Petersen, Christian Borgelt, Hilde Kuehne, and Oliver Deussen. Deep differentiable sorting networks. *International Conference on Learning Representations (ICLR)*, 2022.
- [44] Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 4095–4104, 2018.
- [45] Erwan Plantec, Joachim W. Pedersen, Milton L. Montero, Eleni Nisioti, and Sebastian Risi. Evolving self-assembling neural networks: From spontaneous activity to experience-dependent learning. *arXiv preprint arXiv:2406.09787*, 2024.

- [46] Simon-Shlomo Poil, Richard Hardstone, Huibert D. Mansvelder, and Klaus Linkenkaer-Hansen. Network structure influences self-organized criticality in neural networks with dynamical synapses. *Frontiers in Physics*, 2025.
- [47] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33:4780–4789, 2019.
- [48] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. A comprehensive survey of neural architecture search: Challenges and solutions. *National Science Review*, 11(8):nwae282, 2024. doi: 10.1093/nsr/nwae282.
- [49] Sebastian Risi and Kenneth O. Stanley. An enhanced hypercube-based encoding for evolving the placement, density, and connectivity of neurons. *Artificial Life*, 18(4):331–363, 2012. doi: 10.1162/ARTL_a_00071.
- [50] Maria Rodriguez, Juan Lopez, and Ana Garcia. Evolutionarily optimized network topology as a structural prior for data-efficient sparse neural classification. *bioRxiv*, 2026.
- [51] Nicolás Rodríguez, Pablo Estévez, and Mauricio Vera. Neuromorphic hierarchical modular reservoirs. *Nature Communications*, 2026. Forthcoming.
- [52] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi: 10.1037/h0042519.
- [53] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- [54] Ahmed Said, Mohamed Elhoseny, and El-Sayed M. El-Horbaty. Dynamic scheduling of hybrid flow shop problem with uncertain process time and flexible maintenance using neuroevolution of augmenting topologies. *IET Collaborative Intelligent Manufacturing*, 6(3), 2024.
- [55] Zubair Shah and Noaman Khan. Pruning as evolution: Emergent sparsity through selection dynamics in neural networks. *arXiv preprint arXiv:2601.10765*, 2026.
- [56] Dhadkan Shrestha and Damian Valles. Reinforced NEAT algorithms for autonomous rover navigation in multi-room dynamic scenario. *Fire*, 8(2):41, 2025. doi: 10.3390/fire8020041.
- [57] Hava T. Siegelmann and Eduardo D. Sontag. On the computational power of neural nets. *Journal of Computer and System Sciences*, 50(1):132–150, 1995. doi: 10.1006/jcss.1995.1013.
- [58] Kai-Yeung Siu, Jehoshua Bruck, Thomas Kailath, and Thomas Hofmeister. Depth efficient neural networks for division and related computations. *IEEE Transactions on Information Theory*, 39(3):945–956, 1993. doi: 10.1109/18.256504.
- [59] Kai-Yeung Siu, Vwani Roychowdhury, and Thomas Kailath. *Discrete Neural Computation: A Theoretical Foundation*. Prentice Hall, 1995.
- [60] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems (NIPS 2012)*, volume 25, pages 2951–2959, 2012.

- [61] Kenneth O. Stanley and Risto Miikkulainen. Efficient reinforcement learning through evolving neural network topologies. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2002)*, pages 9–16, 2002.
- [62] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002. doi: 10.1162/106365602320169811.
- [63] Kenneth O. Stanley, David B. D’Ambrosio, and Jason Gauci. A hypercube-based encoding for evolving large-scale neural networks. *Artificial Life*, 15(2):185–212, 2009. doi: 10.1162/artl.2009.15.2.15202.
- [64] John Stogin, Ankur Mali, and C. Lee Giles. A provably stable neural network turing machine with finite precision and time. *Information and Computation*, 658:120034, 2024. doi: 10.1016/j.ic.2024.105168.
- [65] Rainer Storn and Kenneth Price. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997. doi: 10.1023/A:1008202821328.
- [66] Yujin Sun, Rui Wang, and Chen Li. Modular growth of hierarchical networks: Efficient, general, and robust curriculum learning. In *ALIFE 2024: Proceedings of the 2024 Artificial Life Conference*, 2024. doi: 10.1162/isal_a_00780.
- [67] Zoltán Toroczkai and Kevin E. Bassler. Network dynamics: Jamming is limited in scale-free systems. *Nature*, 428:716, 2004. doi: 10.1038/428716a.
- [68] Cleber A. Trujillo, Runjie Gao, Priscilla D. Negraes, Jing Gu, Justin Buchanan, Sebastian Preissl, Allen Wang, Wei Wu, Gabriel G. Haddad, Isaac A. Chaim, Alain Domissy, Matthieu Vandenberghe, Anna Devor, Gene W. Yeo, Bradley Voytek, and Alysson R. Muotri. Complex oscillatory waves emerging from cortical organoids model early human brain network development. *Cell Stem Cell*, 2019.
- [69] Lishuang Wang, Mengfei Zhao, Enyu Liu, and Ran Cheng. Tensorized neuroevolution of augmenting topologies for GPU acceleration. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO ’24)*, pages 1156–1164, 2024. doi: 10.1145/3638529.3654152.
- [70] Sheng-Jun Wang, Claus C. Hilgetag, and Changsong Zhou. Sustained activity in hierarchical modular neural networks: Criticality and optimality. *Physical Review E*, 2011.
- [71] Xiao-Jing Wang. Synaptic reverberation underlying mnemonic persistent activity in prefrontal cortex. *Journal of Neuroscience*, 21(21):RP177, 2001.
- [72] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893.
- [73] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2017. ICLR 2017.